AOI: Turning Failed Trajectories into Training Signals for Autonomous Cloud Diagnosis

Pei Yang[1*] Wanyi Chen[2*] Asuka Yuxi Zheng[3*] Xueqian Li[4] Xiang Li[5] Haoqin Tu[3] Jie Xiao[1] Yifan Pang[6] Dongdong Zhang[7] Fuqiang Li[8] Alfred Long[1] Lynn Ai[1] Eric Yang[1] Bill Shi[1†]

[1]Gradient [2]Soochow University [3]UC Santa Cruz [4]Georgia Institute of Technology
[5]University College London [6]Cookiy.ai [7]WeJoy [8]ByteDance
[*]Equal contribution [†]Corresponding author

arXiv:2603.03378v4 [cs.LG] 16 Oct 2026

Large language model (LLM) agents offer a promising data-driven approach to automating Site Reliability Engineering (SRE), yet their enterprise deployment is fundamentally constrained by three challenges: restricted access to proprietary operational data, unsafe action execution under permission-governed environments, and the inability of closed systems to improve from failure trajectories.

We present AOI (Autonomous Operations Intelligence), a trainable multi-agent framework that formulates automated operations as a structured trajectory learning problem under security constraints. Our approach integrates three key components. First, we introduce a **trainable diagnostic system** that applies Group Relative Policy Optimization (GRPO) to distill expert-level operational knowledge into locally deployed open-source models, enabling preference-based learning without exposing sensitive data. Second, we design a **read–write separated execution architecture** that decomposes operational trajectories into observation, reasoning, and action phases, allowing safe learning over diagnostic traces while preventing unauthorized state mutation. Third, we propose a **Failure Trajectory Closed-Loop Evolver** that mines unsuccessful diagnostic trajectories and converts them into corrective supervision signals, enabling continual data augmentation and distributional refinement within a closed environment.

Evaluated on the AIOpsLab benchmark, our contributions yield cumulative gains across three evaluation dimensions. (1) The AOI runtime alone—without any task-specific training—achieves **66.3%** best@5 success on all 86 tasks, outperforming the prior state-of-the-art (41.9%) by **24.4 percentage points**. (2) Adding Observer GRPO training, a locally deployed 14B model reaches **42.9%** avg@1 on 63 held-out tasks with unseen fault types, surpassing Claude Sonnet 4.5 (41.3%) without multi-run sampling. (3) The Evolver further converts 37 previously failed trajectories into diagnostic guidance, improving end-to-end avg@5 by **4.8 percentage points** while reducing run-to-run variance by **35%**.

Date: Mar 16, 2026
Correspondence: tianyu@gradient.network
Project Page: https://github.com/OpenEdgeHQ/aoi

1 Introduction

Site Reliability Engineering (SRE) has become the backbone of modern digital infrastructure, ensuring the stability that users rely on. As these systems grow in complexity, the drive for efficiency naturally accelerates, pushing teams toward automation to minimize downtime Beyer et al. [2016]. Large Language Model (LLM) agents offer a promising path toward automation, as recent work demonstrates that LLMs can use tools Schick et al. [2023], Weng [2023], execute multi-step plans via chain-of-thought reasoning Wei et al. [2022], Yao et al. [2023], and interact with complex system environments Chen et al. [2025a,b]. However, in enterprise deployments, the security of the agent becomes the defining factor for its adoption, which includes data

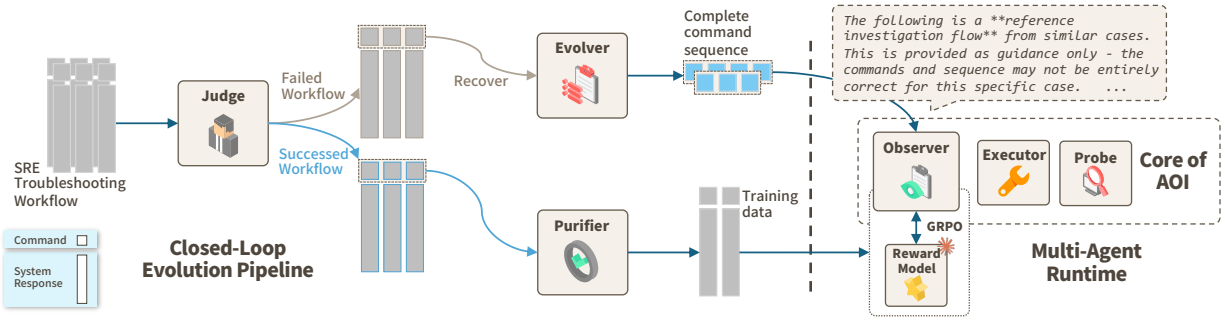

Figure 1. AOI System Overview. Left: *Closed-Loop Evolution Pipeline*—a Judge classifies SRE troubleshooting workflows by outcome. Failed workflows are repaired by the Evolver into corrected command sequences that serve as diagnostic guidance at inference time. Successful workflows are distilled by the Purifier into optimal diagnostic paths that serve as training data. Right: *Multi-Agent Runtime*—the Observer coordinates read-only diagnosis (Probe) and write-gated remediation (Executor). The Observer is trained via GRPO, and at inference time receives the Evolver's corrected plans as structured prompts.

privacy, permission boundaries, and execution safety. Based on these two, we phrase them as challenges that hold back autonomous SRE agents in real-world enterprise operating systems.

Deploying agents in enterprise environments presents a dual challenge of security and adaptability. (1) Strictly governed operational protocols demand a clear separation between diagnostic (or "read") and remediation (or "write") privileges of the system, yet standard automation often conflates these rights, proposing the need of a system that enforces granular execution permissions only when strictly necessary. (2) Compounding this complexity, data privacy specifically in the SRE environment mandates force reliance on smaller, locally deployed models (<100B)[1] that lack expert reasoning; these constrained systems remain fragile and static, failing to take advantage of valuable learning signals from diagnostic failures to adapt to evolving cloud environments.

To meet with these requirements, we propose **AOI** (Autonomous Operations Intelligence), designed to address both challenges through two integrated mechanisms. (1) To address the first challenge that current systems mix the "read" from "write" permissions in the autonomous setup, instead of deploying simple prompt engineering on a single LLM, we introduce a multi-agent system Wu et al. [2023], Hong et al. [2023]comprising three parts: Observer, Probe, and Executor that strictly separates the "read" and "write" actions. In detail, high-risk "write" commands are technically isolated and can only be triggered after sufficient evidence is gathered and verified, aligning the agent's behavior with the principle of least privilege Saltzer and Schroeder [1975]. (2) Building upon this agent architecture, we propose a trainable and evolving system designed to elevate the performance of our AOI beyond the limitations of static deployments. Specifically, we employ Group Relative Policy Optimization (GRPO) Shao et al. [2024] to fine-tune the Observer component (i.e., a 14B open-weight LLM), distilling expert-level diagnostic capabilities Hinton et al. [2015] that bridge the gap between small-scale models and proprietary alternatives. To facilitate continuous learning during model inference, we introduce an Evolver dedicated to Failure Trajectory collection, which allows the Observer to systematically learn from its own operational experiences. This internal feedback mechanism serves to recycle failed diagnostic trajectories into high-quality corrective training signals, fostering a self-sustaining cycle where the agent continuously evolves. By doing so, the system progressively improves its diagnostic precision through real-world interactions while maintaining the strict data isolation required by real environments. By leveraging our agentic architecture and training pipeline, the 14B model approaches Claude Sonnet 4.5 on the AIOpsLab benchmark. This result validates the effectiveness of our AOI in enabling small, locally deployable models to close the gap with frontier models.

Extensive empirical evaluations on the AIOpsLab benchmark substantiate the efficacy of our design. First, the AOI runtime alone—without any task-specific training—achieves +**24.4%** higher success rate in the best@5

[1] A 100B-parameter model under FP16 requires ∼200 GB for weights alone; adding KV cache for 14K-token context (∼10 GB) and system overhead brings the total to ∼215 GB, far exceeding the deployment cost budget of most enterprises.

setting compared with the prior state-of-the-art (41.9% vs. 66.3%), demonstrating that read-write separation yields immediate architectural gains. Second, Observer GRPO training on only 23 tasks generalizes to 63 held-out tasks spanning unseen fault types, lifting avg@1 from 33.7% to **42.9%**—surpassing Claude Sonnet 4.5 (41.3%) without multi-run sampling. Third, the Evolver converts all 37 previously failed trajectories (43% of the benchmark) into structural diagnostic prompts, improving end-to-end avg@5 by **4.8%** while reducing run-to-run variance by **35%**. These results confirm that enforcing strict security constraints and closing the data loop not only ensures safety but also drives robust, reproducible operational capability.

2 Related Work

LLM Agents for AIOps. The application of LLMs to cloud operations spans log analysis Guo et al. [2021], anomaly detection Zhang et al. [2024], root cause analysis Wang and Qi [2024], Chen et al. [2024], and interactive diagnosis. STRATUS Chen et al. [2025b] employs multi-agent collaboration but tightly couples reasoning with execution, causing safety issues and brittle long-horizon behavior. Concurrent work explores retrieval-augmented diagnosis Lewis et al. [2020] and chain-of-thought prompting Wei et al. [2022] for RCA. AOI differs by *architecturally enforcing* safety through role separation rather than relying on prompt-based guardrails.

Safe Agentic Systems. Safety mechanisms for LLM agents include action filtering Schick et al. [2023], Weng [2023], sandboxed execution environments Jimenez et al. [2024], Yao et al. [2023], and reinforcement learning from human feedback Ouyang et al. [2022]. These approaches treat safety as an add-on constraint. We instead design safety into the system architecture: the Observer *cannot* directly execute commands; the Probe *cannot* mutate state; the Executor operates under strict whitelists. This separation-of-concerns approach is inspired by classic operating system security principles Saltzer and Schroeder [1975].

Learning from Failures. Failure analysis has long been central to software engineering Gupta et al. [2017]. Recent work uses failed test cases to guide program repair Vasic et al. [2019], while Reflexion Shinn et al. [2023] enables LLM agents to learn from verbal feedback on failed attempts. In reinforcement learning, hindsight experience replay Andrychowicz et al. [2017] relabels failed trajectories with achieved goals. Our approach differs: we use failed diagnostic *command sequences* as input to a corrective model that generates improved plans. GRPO Shao et al. [2024] provides critic-free optimization suitable for tasks with multiple valid corrections, following the broader trend of RL-based LLM alignment Ouyang et al. [2022], Schulman et al. [2017], Rafailov et al. [2023], DeepSeek-AI [2025].

3 AOI Runtime Architecture

The AOI runtime agent system embodies three design principles: (1) **Safety through Separation**—read-only diagnosis decoupled from state mutation, following the principle of least privilege Saltzer and Schroeder [1975]; (2) **Context Efficiency**—verbose outputs compressed while preserving critical evidence, addressing the well-known context degradation in long inputs Liu et al. [2024]; (3) **Long-Horizon Coherence**—dual-timescale memory Packer et al. [2023]maintains hypotheses across iterations. Figure 2 illustrates the runtime architecture.

3.1 Agent Components and Permissions

AOI employs four specialized agents in a structured multi-agent architecture Du et al. [2024] (Table 1):

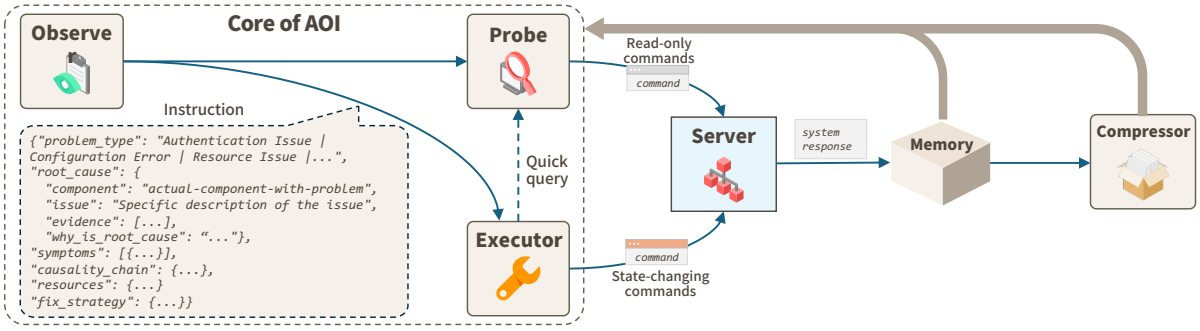

Figure 2. AOI Runtime Agent Architecture. The Observer coordinates diagnosis through the Probe (read-only) and Executor (write-gated) agents. The Compressor maintains context efficiency via dual-timescale memory.

Table 1. Agent roles and permissions in AOI. Right columns show the access control matrix over three memory stores (R = read, W = write, - = no access): $\mathcal{M}_{raw}$ stores raw environment outputs (e.g., full kubectl responses); $\mathcal{M}_{task}$ maintains the diagnostic task queue and hypothesis state; $\mathcal{M}_{comp}$ holds compressed context produced by the Compressor, serving as the Observer's sole information source.

Agent	Responsibility	Permission	$\mathcal{M}_{raw}$	$\mathcal{M}_{task}$	$\mathcal{M}_{comp}$
Observer ($\mathcal{O}$)	Planning, hypothesis tracking	Read compressed	-	R/W	R
Probe ($\mathcal{P}$)	Diagnostic exploration	Read-only exec	W	R	-
Executor ($\mathcal{X}$)	Remediation actions	Write (gated)	W	R	-
Compressor ($\mathcal{C}$)	Context distillation	Read raw, write comp.	R	-	W

Observer ($\mathcal{O}$) serves as the central coordinator. It maintains a diagnostic task queue, analyzes compressed evidence to update hypotheses, and dispatches either Probe or Executor based on current diagnostic state. Critically, the Observer *never directly interacts with the environment*—it only reasons about what to do next.

Probe ($\mathcal{P}$) handles all read-only operations: `kubectl get`, `describe`, `logs`, and similar commands. It supports multi-round exploration within a single iteration (up to K_{max} rounds) and maintains a baseline context cache to avoid redundant queries. Probe implements retry mechanisms for transient failures (network timeouts, API rate limits).

Executor ($\mathcal{X}$) manages state-altering actions. Before executing, it can invoke a single-round Probe for verification ("look before you leap"). All commands pass through a whitelist filter. Executor implements two-stage error recovery: first analyze the failure, then generate a corrected command. All actions are logged for audit.

Compressor ($\mathcal{C}$) bridges raw outputs and decision-making. It applies rule-based deduplication (collapsing repeated log lines) followed by LLM-based semantic compression. Since transformer-based models Vaswani et al. [2017] suffer from degraded attention to middle-context information Liu et al. [2024], the Compressor uses a sliding window strategy for outputs exceeding context limits and enforces strict token budgets per iteration.

Safety emerges from strict access control between agents and memory stores: Key invariants: (1) Observer cannot read raw outputs—it only sees what Compressor produces, preventing information overload and ensuring consistent inputs; (2) Probe and Executor write to raw store but cannot read it—all information flows through Compressor; (3) Compressor is stateless, processing each iteration independently to avoid error accumulation.

3.2 Runtime Pipeline

3.2.1 Execution Pipeline

Each iteration follows a four-stage pipeline:

1. **Decision**: Observer analyzes (H_{n-2}, C_{n-1}), produces action type $a_n \in \{\text{Probe}, \text{Execute}, \text{Submit}\}$ and instruction I_n.
2. **Interaction**: Platform routes to appropriate agent; agent executes commands in environment $\mathcal{E}$ and writes raw outputs to $\mathcal{M}_{raw}$.
3. **Compression**: Compressor processes raw outputs into C_n within token budget.
4. **Caching**: Platform stores C_n for next iteration; Observer produces summary S_{n-1} appended to H.

This pipeline ensures "raw evidence → compression → decision input," enforcing budgets at each stage.

3.2.2 Dual-Timescale Memory

Long-horizon diagnosis requires maintaining coherence across iterations while respecting context limits. We achieve this through dual-timescale memory: **Long-term memory** stores semantic summaries of all past iterations: $H_{n-2} = \{S_1, S_2, \ldots, S_{n-2}\}$. Each summary S_i captures the key findings and hypothesis updates from iteration i. **Short-term memory** contains the full compressed context from the previous iteration: $C_{n-1} = \text{Compress}(\text{RawOutputs}_{n-1})$.

At iteration n, the Observer receives (H_{n-2}, C_{n-1})—historical context for continuity plus recent details for informed decisions. This design bounds memory growth while preserving reasoning coherence.

3.3 Observer Step-Level Policy Optimization

AOI applies GRPO at two distinct granularities to train two different components. We briefly contrast them before detailing the Observer's formulation:

- **Observer GRPO** (this section) optimizes *step-level decisions*: at each diagnostic iteration, the Observer must choose what action to take next and generate the corresponding context. The reward evaluates whether a single decision effectively advances diagnosis given the current evidence.
- **Evolver GRPO** (Section 4) optimizes *trajectory-level generation*: given a complete seed trajectory, the Evolver produces an improved command sequence covering the entire diagnostic workflow. The reward evaluates the quality of the complete output trajectory.

Both share the same GRPO algorithmic framework—sampling G candidates per input and computing group-normalized advantages—but differ in three key aspects: (1) **optimization granularity** (single decision vs. complete trajectory); (2) **reward design** (six-dimensional step quality vs. four-dimensional trajectory quality); and (3) **training objective** (learning diagnostic reasoning patterns vs. learning to augment and repair command sequences). We detail the Observer formulation below; the Evolver formulation follows in Section 4.

3.3.1 GRPO Formulation

For diagnostic tasks, multiple actions can be correct at each step—checking pod logs vs. describing deployments may both yield useful information. Unlike standard PPO Schulman et al. [2017] which requires a learned value function, or DPO Rafailov et al. [2023] which requires pairwise preference data, we optimize the Observer with GRPO Shao et al. [2024] using within-group comparisons rather than absolute rewards.

The training data is derived from successful trajectories that are first processed by a Purifier agent (Figure 1), which strips redundant commands (retries, dead-end explorations) and retains only the minimal command sequence leading to correct diagnosis. For each observation context x (comprising compressed evidence and task state), we sample a group of G candidate actions $\{y_i\}_{i=1}^{G}$ from the Observer policy. Each candidate is scored by an LLM judge $R(x, y_i) \in [0, 1]$. We compute group-normalized advantages:

$$A_i = \frac{R(x, y_i) - \mu_G}{\sigma_G + \epsilon}, \quad \mu_G = \frac{1}{G} \sum_{j=1}^{G} R(x, y_j) \tag{1}$$

The policy gradient update maximizes expected advantage:

$$\nabla_\theta J = \mathbb{E}_{x,\{y_i\}} \left[\sum_{i=1}^{G} A_i \nabla_\theta \log \pi_\theta(y_i|x) \right] \tag{2}$$

The complete training procedure is detailed in Algorithm 2 (Appendix A.2).

3.3.2 Multi-Dimensional Reward Function

Unlike the Evolver, which evaluates complete corrected trajectories, the Observer reward operates at the *step level*: at each iteration n, an LLM judge Zheng et al. [2023] scores the Observer's decision output across six dimensions. The total reward is a weighted sum of normalized dimension scores:

$$R(x, y) = \sum_{d \in \mathcal{D}} w_d \cdot \frac{s_d}{10}, \quad \sum_{d \in \mathcal{D}} w_d = 1 \tag{3}$$

where $s_d \in [0, 10]$ is the raw score for dimension d and w_d is its weight. The six dimensions and their default weights are:

- **Format** (w=0.10, rule-based): Whether the output is valid JSON. Single clean output scores 10; multiple/repeated JSON blocks score 5; parse failure scores 0 and triggers a hard penalty (R=0.09).
- **Summary** (w=0.15, LLM): Accuracy of the previous-iteration summary—does it cite specific error messages, component names, and status rather than vague speculation?
- **Action** (w=0.10, LLM): Correctness of the next action type (Probe / Executor / Submit) against the ground-truth action.
- **Context Instruction** (w=0.30, LLM): Quality of the diagnostic reasoning in probe or executor context—does it logically follow from execution history and effectively advance diagnosis?
- **Context Namespace** (w=0.30, LLM): Accuracy of targeted resources (namespaces, pods, services)—are they the right direction for solving the problem at this stage?

- **Confidence** (w=0.05, LLM): Calibration of the self-reported confidence score relative to the iteration stage (e.g., high confidence at iteration 1 is penalized).

Context Instruction and Context Namespace together account for 60% of the total weight, reflecting our emphasis on *diagnostic reasoning quality* and *target accuracy*—the two factors that most directly determine information gain per step.

4 Trajectory Evolver

While Observer training improves step-level decision-making, a trainable multi-agent system faces a fundamental data scarcity paradox: (1) tasks the system already solves yield trajectories of limited training value—the model gains little from rehearsing what it can already do; (2) tasks the system fails on represent the most valuable improvement targets, yet failed trajectories cannot serve directly as positive examples; (3) high-quality external data (e.g., expert SRE runbooks Beyer et al. [2016]) is scarce and expensive to obtain. The Trajectory Evolver resolves this paradox through two mechanisms: *augmenting* successful trajectories into diverse diagnostic workflow variants, and *repairing* failed trajectories into corrected plans that provide directional guidance for future attempts.

4.1 Problem Formulation

Given any seed trajectory $\tau = (c_1, r_1, \ldots, c_T, r_T)$, the Evolver learns a policy π_{evolve} that generates an improved command sequence:

$$\tau^* = \pi_{\text{evolve}}(\cdot \mid \tau, \text{problem}) \tag{4}$$

For failed seeds (τ^-), this constitutes *repair*: the Evolver corrects diagnostic errors while preserving valid reasoning steps. Failed trajectories provide contrastive supervision—we know what does not work, and often the failures are "near-misses" that correctly identify the faulty component but apply incorrect remediation. For success seeds (τ^+), this constitutes *augmentation*: the Evolver generates alternative diagnostic workflows that achieve the same goal through different command sequences, expanding training diversity from limited expert data.

4.2 Seeds: Definition and Data Source

In the Evolver's context, a **seed** is a complete fault diagnosis command-sequence trajectory $\tau = (c_1, r_1, \ldots, c_T, r_T)$, spanning from initial exploration to final submission. Seeds are categorized by outcome:

- **Success seeds**: Trajectories that correctly resolved the incident, containing a validated diagnostic path.
- **Failed seeds**: Trajectories that did not resolve the incident, but still contain partial diagnostic value (e.g., correct fault localization with incorrect remediation).

The Evolver treats two seed types differently:

- **Augmentation** (success → diverse variants): From a single successful trajectory, the Evolver generates multiple command-sequence variants that preserve the core diagnostic logic while varying command choices,

exploration order, and supplementary steps. This multiplies limited expert data into diverse training corpus.

- **Repair** (failed → corrected plans): Given full context (problem description + failed trajectory + reference success from similar fault types), the Evolver generates a corrected diagnostic plan. These outputs are not guaranteed to solve the problem directly, but provide the correct *diagnostic reasoning flow*—the right investigation direction, command structure, and remediation strategy—that guides the multi-agent system toward success on retry.

In production SRE environments, historical incident records from human operators are a natural seed source—they represent validated diagnostic workflows accumulated over operational history. In our experiments, we use Claude Sonnet 4.5 trajectories on AIOpsLab as a proxy for such expert records. This choice is deliberate: seeds represent *expert-level diagnostic knowledge*, not the multi-agent system's own outputs. The Evolver's design is agnostic to seed provenance—seeds from human SRE runbooks, frontier models, or the system's own historical successes are interchangeable without any architectural change. We use Sonnet 4.5 solely because it provides sufficient high-quality seeds on the benchmark. For GRPO training, we use only success seeds to ensure training data correctness; for inference, both seed types serve as input (details in Section 5).

4.3 GRPO-Optimized Trajectory Correction

We optimize the Evolver using Group Relative Policy Optimization (GRPO), which is well-suited for tasks where multiple corrected plans may be valid.

For each failed trajectory τ^-, sample G candidate corrections $\{\tau_i^+\}_{i=1}^G$. Score each correction using a reward model that evaluates:

- **Validity**: Is the corrected plan executable?
- **Completeness**: Does it cover necessary diagnostic steps?
- **Correctness**: Are the commands syntactically and semantically correct?
- **Effectiveness**: Would this plan lead to successful diagnosis?

Compute group-normalized advantages:

$$A_i = \frac{R(\tau^-, \tau_i^+) - \mu_G}{\sigma_G + \epsilon}, \quad \mu_G = \frac{1}{G}\sum_j R(\tau^-, \tau_j^+) \tag{5}$$

GRPO's critic-free design Shao et al. [2024], DeepSeek-AI [2025] avoids overfitting to a single "correct" diagnostic path, preserving diversity in valid approaches.

4.4 Integration with AOI system

Figure 5 (Appendix) illustrates the Evolver's integration with the AOI runtime.

The corrected command list from the Evolver is provided to the Observer as a structured prompt:

```
[Corrected Diagnostic Plan]
```

```
Based on analysis of the failed attempt,
the following commands should be executed:
1. kubectl get pods -n {namespace}
2. kubectl describe pod {pod-name}
...
```

This integration allows the Observer to benefit from learned corrections while retaining flexibility to adapt based on actual system responses. The Evolver provides *guidance*, not rigid constraints.

5 Experiments

5.1 Experimental Setup

We evaluate on AIOpsLab Chen et al. [2025a], containing 86 incident scenarios across three microservice applications on live Kubernetes Burns et al. [2016] clusters. Tasks span four categories: **Detection** (32), **Localization** (28), **RCA** (13), and **Mitigation** (13). We compare against: (1) **AOL-agent** Chen et al. [2025a], the benchmark's reference ReAct-style Yao et al. [2023] agent; (2) **STRATUS** Chen et al. [2025b], a multi-agent system with specialized detection/diagnosis/mitigation modules; and (3) **Claude Sonnet 4.5** Anthropic [2025] with AOL-agent, whose trajectories serve as seed data for our training pipeline.

5.1.1 Data Split

We adopt a nested partition structure illustrated in Figure 6(Appendix). All three sets share the same execution environment, ensuring consistency across training and evaluation.

- $\mathcal{D}^{all}$: the full benchmark. Runtime comparison is evaluated over all 86 tasks using the base model *without any task-specific training*, ensuring a fair architectural comparison against STRATUS. The training subsets below are drawn from $\mathcal{D}^{all}$.
- $\mathcal{D}^{train}_{evolver}$: 49 tasks successfully solved by Claude Sonnet 4.5, serving as success seeds for Evolver GRPO training.
- $\mathcal{D}^{train}_{obs}$: a strict subset of $\mathcal{D}^{train}_{evolver}$ covering 11 fault types and 23 entries, used for Observer GRPO training.
- $\mathcal{D}^{test}_{obs}$: 63 held-out evaluation examples for the Observer, covering 15 unseen fault types plus tasks from training fault types where Sonnet failed.
- $\mathcal{D}^{test}_{evolver}$: 37 failed trajectories from Claude Sonnet 4.5, a subset of $\mathcal{D}^{test}_{obs}$, used to evaluate the Evolver's repair capability.

In the combined pipeline, the Observer receives diagnostic guidance generated by the Evolver. To evaluate this end-to-end effect without data leakage, *every test case must be unseen by both components*. By making the Observer's training set a strict subset of the Evolver's, we guarantee that $\mathcal{D}^{test}_{evolver} \subset \mathcal{D}^{test}_{obs}$—all 37 Evolver test cases fall within the Observer's 63 held-out tasks. This nested design enables fair evaluation of each component independently and in combination.

Strict fault-type split for $\mathcal{D}^{train}_{obs}$. We adopt the most stringent split strategy: training and test fault types have **zero overlap**. The 86 tasks span 26 distinct fault types; we assign 11 types (38 total tasks, of which 23

Table 2. AOI runtime comparison on full benchmark (86 tasks, %). GPT-4o-mini OpenAI [2023], Claude Sonnet 4.5 uses single-run; Qwen3-14B uses 5-round sampling (best@5/avg@5).

Model	Method	Detection	Localization	RCA	Mitigation	Overall
GPT-4o-mini	AOL-agent	25.0	9.5	7.7	7.7	14.7
GPT-4o-mini	STRATUS	78.1	25.0	15.4	23.1	43.0
GPT-4o-mini	AOI	**90.6**	**32.1**	**38.5**	**53.8**	**58.1**
Claude Sonnet 4.5	AOL-agent	68.8	53.6	15.4	76.9	57.0
Qwen3-14B (best@5/avg@5)	STRATUS	75.0/41.3	32.1/11.4	7.7/4.6	15.4/15.4	41.9/22.1
Qwen3-14B (best@5/avg@5)	AOI	**100/66.9**	**53.6/27.9**	**30.8/7.7**	**46.2/23.1**	**66.3/38.6**

have successful Sonnet trajectories) to training and reserve the remaining 15 types exclusively for testing. This ensures that the Observer is evaluated on genuinely novel fault categories, not merely unseen instances of familiar faults. The complete fault-type partition is provided in Appendix B.3.

5.1.2 Metrics.

We report success using two standard multi-run aggregation metrics: **best@**k, where a task is counted as solved if any of k independent runs succeeds; and **avg@**k, the mean success rate across k runs.

5.1.3 Implementation.

Our method is built on Qwen3-14B Yang et al. [2025] as the base model. We apply LoRA Hu et al. [2022] fine-tuning with rank 64, α=128, and learning rate 10^{-5}; GRPO Shao et al. [2024] is configured with group size G=4. All experiments run on 2×A100 GPUs with vLLM Kwon et al. [2023] for inference.

5.2 AOI Runtime vs Baseline

We compare AOI against STRATUS on $\mathcal{D}^{all}$ (86 tasks) with identical base models. No task-specific training.

Architecture yields 4× improvement over vanilla agents. AOI with GPT-4o-mini achieves 58.1% overall compared to AOL-agent's 14.7%. The gains stem from read-write separation: the Observer can safely explore diagnostic paths without risking state mutations, while the Executor applies changes only after sufficient evidence accumulates.

Task complexity reveals architectural bottlenecks. Detection benefits most from safe exploration (100% best@5 vs STRATUS's 75%), as agents can issue multiple diagnostic commands without penalty. RCA shows the largest relative gains (+150% over STRATUS), indicating that AOI's dual-timescale memory helps maintain reasoning coherence across long diagnostic chains. Mitigation improves 3× over STRATUS because Executor-level safety gates prevent the cascading failures that occur when agents attempt remediation before completing diagnosis.

Frontier models excel at pattern-matching, not reasoning. Claude Sonnet 4.5 achieves 76.9% on Mitigation—the highest single-category result—but only 15.4% on RCA, matching GPT-4o-mini + STRATUS. This asymmetry reveals that Sonnet's strength lies in recognizing remediation patterns (restart pods, scale replicas, rollback deployments) rather than multi-step causal reasoning. Mitigation tasks have well-defined action templates once the fault is identified; RCA requires synthesizing signals across logs, metrics, and

Table 3. Observer GRPO on held-out fault types (63 tasks, %).

Method	Det.↑ (22)	Loc.↑ (22)	RCA↑ (12)	Mit.↑ (7)	Overall↑ (63)
Sonnet 4.5 (AOL-agent)	54.5	40.9	8.3	57.1	41.3
AOI (Untrained)	65.5	22.7	6.7	14.3	33.7
AOI (Observer-GRPO)	**90.9**	18.2	**16.7**	14.3	**42.9**

traces to establish causality. AOI's structured diagnostic workflow addresses this gap: with Qwen3-14B, RCA doubles to 30.8% while Mitigation reaches 46.2%.

Open-weight surpasses frontier with proper architecture. Qwen3-14B + AOI (66.3% best@5) outperforms Claude Sonnet 4.5 + AOL-agent (57.0%), solving 57 vs 49 tasks. The 14B model's disadvantage in raw capability is offset by architectural support for systematic exploration and safe execution. Diminishing returns analysis (Figure 7, Appendix) shows best@1→2 gains +19.8 points while best@3→5 adds only +8.2 combined, suggesting 2-3 sampling rounds capture most benefit.

5.3 Observer GRPO Generalization

We evaluate whether Observer GRPO training improves generalization to unseen fault types.

5.3.1 Held-out Fault Types

We train Observer on $\mathcal{D}_{obs}^{train}$ (23 tasks) and evaluate on 63 held-out tasks (unseen fault types).

GRPO learns task-completion strategies, not intermediate accuracy. Observer-GRPO achieves 90.9% Detection (+36 points over Sonnet) but Localization drops from 22.7% to 18.2%. This trade-off is inherent to GRPO's reward structure: the model optimizes for end-task success, learning that Detection can succeed with coarse-grained anomaly signals while Localization requires precise component identification. The trained Observer prioritizes high-confidence fault indicators over exhaustive exploration. Appendix E provides comprehensive task-type analysis revealing that GRPO-trained models use ∼9 more exploration steps—beneficial for RCA but harmful for Localization where over-exploration causes multi-anomaly confusion.

Single-run open-source beats frontier model. Observer-GRPO (42.9% avg@1) surpasses Claude Sonnet 4.5 (41.3%) on identical held-out tasks without multi-run sampling. The improvement concentrates in Detection (+36.4 points) and RCA (+8.4 points)—precisely the tasks requiring systematic diagnostic reasoning rather than pattern recognition. Mitigation remains unchanged at 14.3%: in AOI's architecture, remediation commands are generated and executed by the Executor, so Observer optimization cannot directly improve the quality of the final repair actions. Sonnet's Mitigation advantage (57.1%) stems from its stronger base capability in generating correct remediation commands.

Generalization to unseen fault types validates transfer learning. The held-out set contains 15 fault types never seen during training. Observer-GRPO's gains demonstrate that diagnostic patterns transfer across fault categories: the model learns *how* to diagnose (command sequencing, signal prioritization) rather than memorizing fault-specific solutions. Per-task analysis (Figure 10, Appendix) shows consistent improvement across fault types rather than selective memorization.

Table 4. Component ablation on $\mathcal{D}^{test}_{evolver}$ (37 tasks, best / avg %). First three rows use 5 runs; last row uses 4 runs.

Method	Det. (10)	Loc. (13)	RCA (11)	Mit. (3)	Overall (37)
Base	100/50.0	**54**/26.2	27/7.3	0/0	**54**/24.9
Evolver-prompts	90/64.0	**54**/24.6	18/12.7	0/0	49/29.7
Observer-GRPO	90/64.0	38/21.5	**36**/**29.1**	0/0	49/33.5
Observer-GRPO + Evolver-prompts	**100**/72.5	31/19.2	**36**/25.0	0/0	49/**33.8**

5.3.2 Combined System on Failed Cases

We evaluate all component combinations on $\mathcal{D}^{test}_{evolver}$ (37 Sonnet-failed tasks, 5 independent runs per task): Base (untrained Observer), Evolver-prompts (Base + Evolver-generated diagnostic prompts), Observer-GRPO (trained Observer without Evolver), and Observer-GRPO + Evolver-prompts (trained Observer with Evolver-generated prompts).

Observer-GRPO + Evolver-prompts achieves highest avg (33.8%, Table 4), a +8.9 point improvement over Base. The combination is complementary: Observer-GRPO improves RCA (+21.8 points over Base), while Evolver-prompts improves Detection consistency (avg: 50%→72.5%).

Components target different failure modes. Observer-GRPO benefits tasks requiring deep exploration (RCA), while Evolver-prompts benefits tasks requiring structured workflows (Detection). Localization shows minimal gains from either—these tasks require precise fault identification that neither exploration depth nor workflow guidance addresses.

Localization hits a structural ceiling. Best@5 Localization is identical across conditions, and avg@5 barely changes. Localization requires pinpointing the exact faulty component, which depends on real-time environment state that offline-generated plans cannot anticipate. The Evolver improves *what* to investigate but not *where* to look.

Dual-timescale synergy. Observer-GRPO + Evolver-prompts combines both components addressing complementary failure modes: Observer-GRPO improves diagnostic *execution* (fast timescale), while Evolver-prompts improves diagnostic *planning* (slow timescale). Their combination recovers nearly a third of cases where Claude Sonnet 4.5 achieved 0%.

Mitigation remains at 0%. Unlike Detection and RCA, where structural diagnostic patterns generalize across fault types, Mitigation requires fault-specific remediation actions absent from the training distribution.

5.4 Repair Quality of Evolver

5.4.1 Evaluation Strategies

We evaluate the Evolver's ability to repair previously failed cases. We train on 49 successful trajectories and test on 37 cases where Claude Sonnet originally failed.The complete command sequences produced by the Evolver are "adapted" versions of the original trajectories—the repaired command lists may differ from the original AIOpsLab problem context (e.g., adjusted command arguments, alternative exploration paths), making direct replay in the environment infeasible. We therefore adopt two complementary evaluation strategies:

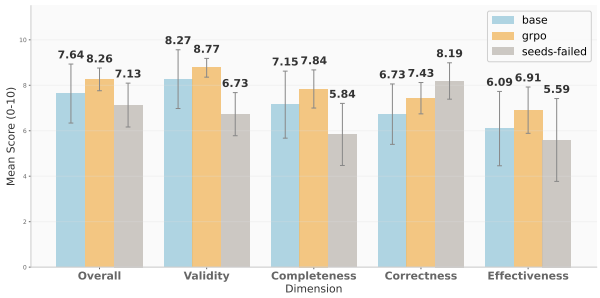

Figure 3. Performance across evaluation dimensions. Seeds-Failed, Base (untrained Qwen3-14B), and GRPO-trained Evolver compared on four reward dimensions and overall score.

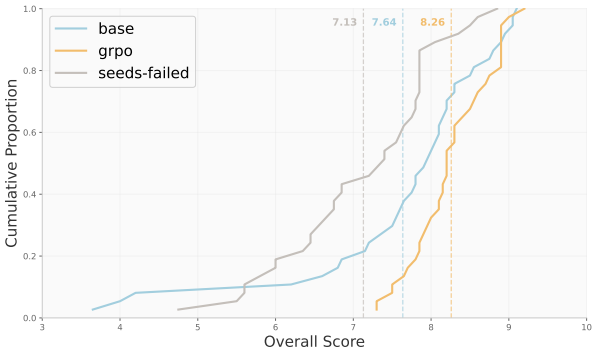

Figure 4. Cumulative distribution of overall scores. Dashed lines indicate group means.

1. **Evolver-as-Prompt (end-to-end validation):** We feed the Evolver's corrected command sequences as prompts to the Observer and measure whether actual task success improves (see Table 4, overall increasing 4.8% on avg@5).
2. **LLM judge scoring (generation quality evaluation):** We use Claude Opus 4.5 to score the Evolver's repaired outputs across four dimensions—Validity, Completeness, Correctness, and Effectiveness—to assess repair quality improvement.

5.4.2 Comparison Experiments

Three groups of experiment are compared in Figure 3 and Figure 4 are:

- **Seeds-Failed**: The original command sequences from Sonnet 4.5's 37 failed interactions with AIOpsLab.
- **Base**: The untrained Qwen3-14B model repairing each of the 37 tasks, generating 2 repair candidates per task.
- **GRPO**: The GRPO-trained Qwen3-14B Evolver repairing each of the 37 tasks using the same Seeds-Failed as prompt input, generating 2 repair candidates per task.

5.4.3 Quality Evaluation Results

In Figure 3, although Seeds-Failed retains higher correctness due to pre-execution filtering, the Evolver models accept minor syntactic variations to achieve diverse diagnostic paths, yielding net gains in Completeness and Effectiveness that outweigh the slight correctness decline. Crucially, improvements in Validity and Completeness indicate that GRPO facilitates structural rather than superficial correction, teaching the Evolver to fill logical gaps in diagnostic reasoning instead of memorizing command strings . This stability is evidenced by the Cumulative Distribution Function (CDF) in Figure 4, where the elimination of the lower-bound tail and a variance reduction (std $0.97 \rightarrow 0.49$) confirm the acquisition of consistent strategies rather than stochastic improvements . By building upon the Base model's non-trivial floor to amplify repair consistency, GRPO proves superior to direct supervised fine-tuning in accommodating diverse valid diagnostic paths .

5.4.4 End-to-End Robustness Analysis (Table 4)

Evolver-prompts improves robustness, not peak performance. The most revealing pattern in Table 4 lies in the gap between best@5 and avg@5—a direct measure of run-to-run variance. For Base, this gap is 29.2pp (54.1%−24.9%); for Evolver-prompts, it shrinks to 18.9pp (48.6%−29.7%). While best@5 decreases slightly (−5.5pp), avg@5 improves meaningfully (+4.8pp). Evolver-prompts' structural prompts act as *diagnostic scaffolding* that constrains the Observer's search space, making successful diagnostic paths reliably reproducible.

Detection: variance halved. Without Evolver prompts, the Observer achieves 100% best@5 but only 50% avg@5—a 50pp gap indicating that half of all runs fail despite the problem being solvable. With Evolver prompts, best@5 drops to 90% but avg@5 jumps to 64%, compressing the gap to 26pp.

Analysis benefits disproportionately from structural guidance. Although Analysis best@5 decreases (27.3%→18.2%), avg@5 nearly doubles (7.3%→12.7%). This divergence is informative: Analysis tasks—root cause identification requiring multi-step reasoning chains—are the most sensitive to diagnostic plan quality. Without prompts, the Observer occasionally stumbles upon correct causal chains (high best@5) but cannot reliably reproduce them (low avg@5). The Evolver's prompts provide a reasoning scaffold that makes successful diagnosis *repeatable* rather than accidental, which is precisely the property needed for production deployment where consistent reliability matters more than occasional brilliance.

6 Discussion

Failed trajectories encode recoverable signal. The conventional wisdom treats failed diagnostic attempts as noise to be filtered. Our experiments suggest otherwise: the GRPO-trained Evolver significantly improves repair quality on previously failed cases (mean LLM judge score 7.18→8.27, std 0.97→0.49) by learning *what was almost right*. Failed trajectories often contain correct diagnostic intuitions paired with execution errors—wrong command flags, incorrect resource names, or missing intermediate steps. GRPO's contrastive learning surfaces these near-miss patterns, enabling systematic correction rather than wholesale replacement.

Diminishing returns in sampling diversity. Our best@k analysis reveals a surprising pattern: success rate improves by 19.8 points from one to two runs, but only 8.2 points across the subsequent three runs combined (Figure 7). This suggests that diagnostic stochasticity is bounded—most tasks either succeed consistently, fail consistently, or exhibit binary variance that two attempts suffice to capture. For practitioners, this implies that allocating compute budget to 2-3 independent runs yields better cost-effectiveness than deeper single-run optimization.

The speed-precision trade-off in learned diagnostics. Observer GRPO improves Detection by 25.5 points while *degrading* Localization by 4.5 points. Task-level analysis reveals the mechanism: GRPO learns aggressive diagnostic shortcuts that terminate earlier with correct anomaly verdicts, but occasionally skip the detailed exploration steps required for precise fault localization. The two degraded tasks (pod_failure_hotel_res, astronomy_shop_product_catalog) both require multi-hop reasoning through service dependencies—exactly the exploration that fast paths eliminate. This trade-off is not a bug but a feature of reward-driven optimization: the training signal emphasized detection accuracy, and the model responded accordingly.

Safety mechanisms improve capability. Counter-intuitively, constraining the agent's action space *enhances* diagnostic success. Read-write separation forces evidence accumulation before mutation, preventing the cascading failures we observed in STRATUS where premature remediation attempts corrupted system state. This challenges the assumption that safety constraints reduce capability—for operational tasks with

irreversible actions in production environments Burns et al. [2016], Beyer et al. [2016], guardrails enable more aggressive exploration within safe boundaries.

Capability boundaries are task-specific. One-third of tasks (29/86) fail consistently across all configurations—five Qwen rounds, GPT-4o-mini, and GRPO variants. These are not random failures but systematic capability gaps: MongoDB authentication recovery requires Helm-specific knowledge absent from training, and multi-service localization demands causal reasoning beyond current architectures. Importantly, these boundaries are stable and predictable, enabling practitioners to identify which incident types require human escalation.

7 Limitations

Expanding the Evolver. While the current Evolver generates corrected command sequences as structured prompts for LLMs, its role could be extended to produce synthetic system feedback via environment simulators or to serve as a runtime agent for dynamic plan refinement. Such architectural expansions, while promising for further boosting system autonomy, are beyond the scope of this work and are left for future investigation.

Beyond AIOpsLab. Although AIOpsLab offers a realistic evaluation of SRE scenarios, we anticipate the emergence of broader community benchmarks covering more diverse infrastructure stacks and larger-scale deployments. On the applied side, we plan to deploy AOI in production SRE environments to validate the use and productization potential of the framework within real-world incident response workflows.

8 Conclusion

We presented AOI, a system for autonomous cloud incident response built on two innovations: (1) an Observer-Probe-Executor runtime that enforces safety through architectural separation, and (2) a Trajectory-Corrective Evolver that learns to correct failed diagnostic sequences via GRPO optimization. Our results demonstrate a key insight: *failed trajectories are not wasted supervision.* By learning to correct failures rather than discard them, we convert 37 failed cases into training signal that enables systematic capability acquisition. The Evolver improves end-to-end avg@5 by 4.8% while reducing run-to-run variance by 35% (best@5–avg@5 gap: 29.2pp→18.9pp), and the GRPO-trained Evolver achieves higher repair quality (mean LLM judge score 7.18→8.27, std 0.97→0.49), indicating robust learning of correction patterns.

References

Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy, editors. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. URL https://sre.google/sre-book/table-of-contents/.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://arxiv.org/abs/2302.04761.

Lilian Weng. LLM powered autonomous agents. https://lilianweng.github.io/posts/2023-06-23-agent/, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022. URL https://arxiv.org/abs/2201.11903.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629.

Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds. *arXiv preprint arXiv:2501.06706*, 2025a. URL https://arxiv.org/abs/2501.06706.

Yinfang Chen, Jiaqi Pan, Jackson Clark, Yiming Su, Noah Zheutlin, Bhavya Bhavya, Rohan Arora, Yu Deng, Saurabh Jha, and Tianyin Xu. STRATUS: A multi-agent system for autonomous reliability engineering of modern clouds. *arXiv preprint arXiv:2506.02009*, 2025b. URL https://arxiv.org/abs/2506.02009.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. URL https://arxiv.org/abs/2308.08155.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023. URL https://arxiv.org/abs/2308.00352.

Jerome H Saltzer and Michael D Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. URL https://doi.org/10.1109/PROC.1975.9939.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL https://arxiv.org/abs/2402.03300.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. URL https://arxiv.org/abs/1503.02531.

Haixuan Guo, Shuhan Yuan, and Xintao Wu. LogBERT: Log anomaly detection via BERT. In *International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2021.

Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S Yu, and Ying Li. A survey of AIOps for failure management in the era of large language models. *arXiv preprint arXiv:2406.11213*, 2024. URL https://arxiv.org/abs/2406.11213.

Tingting Wang and Guilin Qi. A comprehensive survey on root cause analysis in (micro) services: Methodologies, challenges, and trends. *arXiv preprint arXiv:2408.00803*, 2024. URL https://arxiv.org/abs/2408.00803.

Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, Jun Zeng, Supriyo Ghosh, Xuchao Zhang, Chaoyun Zhang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Tianyin Xu. Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the 19th European Conference on Computer Systems (EuroSys)*, pages 674–688, 2024.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Douwe Kiela, and Veselin Stoyanov. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020. URL https://arxiv.org/abs/2005.11401.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.06770.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 27730–27744, 2022. URL https://arxiv.org/abs/2203.02155.

Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish Shevade. Deepfix: Fixing common C language errors by deep learning. In *AAAI Conference on Artificial Intelligence*, pages 1345–1351, 2017.

Marko Vasic, Aditya Kanade, Petros Maniatis, David Bieber, and Rishabh Singh. Neural program repair by jointly learning to localize and repair. In *International Conference on Learning Representations (ICLR)*, 2019.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://arxiv.org/abs/2303.11366.

Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. URL https://arxiv.org/abs/1707.01495.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://arxiv.org/abs/2305.18290.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. URL https://arxiv.org/abs/2501.12948.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. URL https://arxiv.org/abs/2307.03172.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 11733–11763, 2024. URL https://arxiv.org/abs/2305.14325.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. URL https://arxiv.org/abs/1706.03762.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://arxiv.org/abs/2306.05685.

Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016. URL https://doi.org/10.1145/2890784.

Anthropic. Claude Sonnet 4.5 system card. https://www.anthropic.com/claude-sonnet-4-5-system-card, 2025. Released September 29, 2025.

An Yang, Baosong Yang, Beichen Zhang, Fei Cui, Dayiheng Dai, Huan Ge, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2106.09685.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023. URL https://arxiv.org/abs/2309.06180.

OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. URL https://arxiv.org/abs/2303.08774.

Algorithm 1 AOI Main Loop

1: **Input:** Task $\mathcal{D}$, max iterations N
2: Initialize Observer $\mathcal{O}$, Probe $\mathcal{P}$, Executor $\mathcal{X}$, Compressor $\mathcal{C}$
3: $\mathcal{T} \leftarrow \mathcal{O}.\text{Plan}(\mathcal{D})$ ▷ Initial task queue
4: $H \leftarrow []$ ▷ Long-term memory
5: **for** $n = 1$ to N **do**
6: $(S_{n-1}, a_n, I_n) \leftarrow \mathcal{O}.\text{Decide}(H_{n-2}, C_{n-1}, \mathcal{T})$
7: **if** $n > 1$ **then**
8: $H.\text{append}(S_{n-1})$
9: **end if**
10: **if** $a_n = \text{Submit}$ **then**
11: **return** $\mathcal{E}.\text{Submit}(I_n)$
12: **else if** $a_n = \text{Probe}$ **then**
13: $\mathcal{P}.\text{Run}(I_n, \mathcal{E}) \rightarrow \mathcal{M}_{raw}$
14: **else if** $a_n = \text{Execute}$ **then**
15: $\mathcal{X}.\text{Run}(I_n, \mathcal{E}) \rightarrow \mathcal{M}_{raw}$
16: **end if**
17: $C_n \leftarrow \mathcal{C}.\text{Compress}(\mathcal{M}_{raw}^{(n)})$
18: **end for**
19: **return** $\mathcal{E}.\text{Submit}(\text{timeout})$

Code and Data Availability

Code and pre-trained models: https://anonymous.4open.science/r/aoi-C8C7

Benchmark data. We use the publicly available AIOpsLab benchmark Chen et al. [2025a]. Our fault-type split and trajectory data will be released upon publication.

Note: Anonymous GitHub repository link will be provided for review. The repository will be made public upon paper acceptance.

A Algorithm Details

A.1 AOI Main Loop

Algorithm 1 presents the complete AOI orchestration loop. The design reflects three key principles:

Dual-timescale memory. The algorithm maintains two memory structures: (1) *short-term context* C_n from the Compressor, preserving critical evidence from the current iteration, and (2) *long-term memory* H storing summaries S_{n-1} across iterations. This separation allows the Observer to maintain hypotheses over long diagnostic horizons while keeping per-iteration context within token limits.

Lazy compression. Compression occurs at the end of each iteration (line 17) rather than before decision-making. This ensures the Observer sees raw evidence when making decisions, while only compressed summaries are stored for future iterations—balancing information fidelity with context efficiency.

Action-type routing. The three action types (Submit, Probe, Execute) route to specialized agents with different capabilities and safety constraints, enabling fine-grained access control.

A.2 Observer GRPO Training

Algorithm 2 presents the Observer GRPO training procedure.

Algorithm 2 Observer GRPO Training

Input: Successful trajectories $\mathcal{D}$, group size G, LLM judge R
Output: Trained Observer policy π_θ
for each training step **do**
  Sample context x from $\mathcal{D}$ (compressed evidence + task state)
  Sample G candidate actions $\{y_i\}_{i=1}^{G} \sim \pi_\theta(\cdot|x)$
  for $i = 1$ to G **do**
    $r_i \leftarrow R(x, y_i)$ ▷ LLM judge scoring
  end for
  $\mu_G \leftarrow \frac{1}{G}\sum_i r_i; \quad \sigma_G \leftarrow \sqrt{\frac{1}{G}\sum_i (r_i - \mu_G)^2}$
  $A_i \leftarrow (r_i - \mu_G)/(\sigma_G + \epsilon)$ for all i
  Update θ via policy gradient with advantages $\{A_i\}$
end for

The LLM judge evaluates candidates on six dimensions (consistent with Section 3.3):

- **Format** (w=0.10, rule-based): Whether the output is valid JSON.
- **Summary** (w=0.15, LLM): Accuracy of the previous-iteration summary.
- **Action** (w=0.10, LLM): Correctness of the next action type (Probe / Executor / Submit).
- **Context Instruction** (w=0.30, LLM): Quality of the diagnostic reasoning in probe or executor context.
- **Context Namespace** (w=0.30, LLM): Accuracy of targeted resources (namespaces, pods, services).
- **Confidence** (w=0.05, LLM): Calibration of self-reported confidence relative to iteration stage.

Rationale for dimension weights. Context Instruction and Context Namespace together account for 60% of the total weight, reflecting the emphasis on diagnostic reasoning quality and target accuracy—the two factors that most directly determine information gain per step. Format is rule-based to provide a hard constraint on output validity. Summary, Action, and Confidence serve as auxiliary signals that shape the learning trajectory without dominating the reward.

A.3 Trajectory-Corrective Evolver

The Evolver addresses a key limitation of pure imitation learning: when the Observer encounters situations not covered by successful trajectories, it lacks guidance for recovery. Given a failed trajectory, the Evolver generates candidate corrections via GRPO sampling and selects the highest-scoring correction as a structured prompt for the Observer's next attempt. This creates a closed-loop mechanism that converts failed diagnostic attempts into learning opportunities without requiring manual expert intervention.

Figure 5 illustrates the Evolver's integration with the AOI runtime.

Figure explanation: The Evolver operates in three stages: (1) **Failure Collection** - collects failed trajectories from previous diagnostic attempts; (2) **Correction Generation** - samples multiple candidate corrected command sequences using GRPO-trained policy; (3) **Guidance Injection** - provides the highest-scoring corrected plan as a structured prompt to the Observer for subsequent attempts. The dashed feedback loop shows how this process converts failed trajectories into training signal.

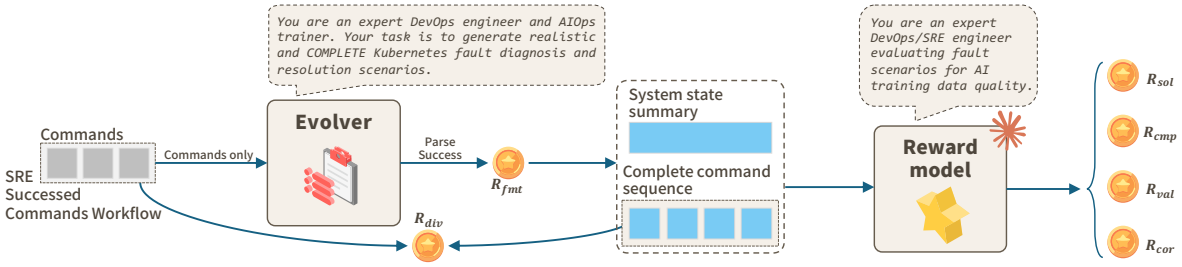

Figure 5. Trajectory-Corrective Evolver Architecture. The Evolver observes failed command sequences from the Observer's execution history, generates G candidate corrections via GRPO sampling (sampling from a policy trained on successful trajectories), and provides the highest-scoring correction as a structured prompt to guide the Observer's next attempt. *Key insight:* This closed-loop mechanism converts failed diagnostic attempts into learning opportunities without requiring manual expert intervention.

B Experimental Setup

B.1 Hyperparameters

Key parameter choices. We set max iterations to 15 based on analysis of successful Claude Sonnet trajectories, where 95% completed within 12 iterations. The context budget (4096 tokens) balances information retention with inference cost—larger budgets showed diminishing returns beyond 4K tokens. For GRPO training, we use LoRA rank 64 (rather than full fine-tuning) to preserve the base model's general capabilities while adapting to diagnostic patterns. The learning rate (10^{-5}) and batch size (16) were selected to ensure stable convergence within 3 epochs.

Table 5. Complete hyperparameter settings.

Parameter	Value
AOI Runtime	
Max iterations	15
Max Probe rounds/iteration	5
Context budget/iteration	4096 tokens
Long-term memory capacity	10 summaries
Executor whitelist	47 command patterns
Trajectory-Corrective Evolver	
Base model	Qwen3-14B
GRPO candidates G	4
Learning rate	1×10^{-5}
LoRA rank / alpha	64 / 128
Batch size	16
Epochs	3
Reward model	Claude Opus 4.5
Training samples 49 success seeds	
Test samples	37 failed seeds

B.2 Benchmark Details

Full benchmark from AIOpsLab Chen et al. [2025a]: 88 scenarios across 3 applications (reduced to 86 in our evaluation due to 2 deprecated scenarios).

Our evaluation split:

- **Success seeds** (49 cases): Claude Sonnet successful trajectories, used as positive examples for GRPO training.
- **Failed seeds** (37 cases): Claude Sonnet failed trajectories that overlap with Observer's held-out set, used as test set to evaluate Evolver's recovery ability.

Failed cases by task type:

Task Type	Count
Detection	10
Localization	13
Mitigation	3
RCA	11
Total	37

Fault types in failed cases include: service failures (31%), misconfigurations (26%), authentication errors (18%), pod failures (15%), and network issues (10%).

B.3 Data Split Details

Rationale for nested data partition. We use a strict subset structure $\mathcal{D}_{obs}^{train} \subset \mathcal{D}_{evolver}^{train} \subset \mathcal{D}^{all}$ to ensure no data leakage: since the Evolver provides guidance to the Observer, any task seen by the Observer during training must also be excluded from Evolver evaluation. This guarantees that all 37 Evolver test cases fall within the Observer's 63 held-out tasks.

Figure 6 and Tables 6–7 detail the fault-type-based partition used for Observer GRPO training.

Table 6. Training fault types (11 types → 23 successful trajectories). These fault types were selected to provide diverse coverage of common failure modes while ensuring sufficient test data remains for evaluation.

Fault Type	GT Tasks	Total Tasks
k8s_target_port-misconfig	9	12
scale_pod_zero_social_net	3	4
misconfig_app_hotel_res	3	4
astronomy_shop_cart_service_failure	1	2
astronomy_shop_payment_service_failure	1	2
astronomy_shop_product_catalog_failure	1	2
astronomy_shop_recommend_cache_failure	1	2
auth_miss_mongodb	1	4
network_loss_hotel_res	1	2
noop_detection_hotel_reservation	1	1
redeploy_without_PV	1	3
Total	**23**	**38**

C Multi-Round Sampling Analysis

Organization note: This section analyzes the stochastic behavior of the AOI system across multiple independent runs. We separate this from the experimental setup (Section B) because it focuses on *empirical characterization* of the system's output distribution rather than configuration details. Understanding variance across runs is critical for deployment planning in production environments.

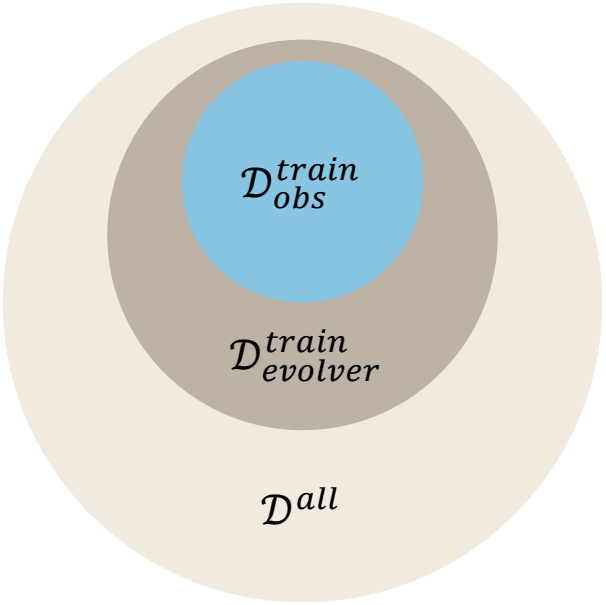

Figure 6. Nested data partition structure. $\mathcal{D}^{all}$ (86 tasks) contains $\mathcal{D}_{evolver}^{train}$ (49 successful trajectories from Claude Sonnet 4.5), which in turn contains $\mathcal{D}_{obs}^{train}$ (23 tasks covering 11 fault types). The strict subset relationship ensures that no test data appears in any training set. *Visual interpretation:* The outer circle represents all benchmark tasks; the middle ring shows Evolver training data (all successful Sonnet trajectories); the innermost core shows Observer training data (carefully selected to cover diverse fault types while maintaining strict hold-out). This design enables fair evaluation of the combined Observer+Evolver system.

Table 7. Test fault types (15 types → 48 tasks) plus 15 tasks from training fault types where Sonnet failed, totaling 63 held-out tasks. *Critical:* All test fault types are completely unseen during Observer training, ensuring we evaluate true generalization capability.

Fault Type	**Tasks**
revoke_auth_mongodb	8
user_unregistered_mongodb	8
assign_to_non_existent_node_social_net	4
wrong_bin_usage	4
astronomy_shop_ad_service_high_cpu	2
astronomy_shop_ad_service_manual_gc	2
astronomy_shop_ad_service_failure	2
network_delay_hotel_res	2
pod_kill_hotel_res	2
pod_failure_hotel_res	2
astronomy_shop_image_slow_load	2
astronomy_shop_kafka_queue_problems	2
astronomy_shop_loadgen_flood_homepage	2
astronomy_shop_payment_unreachable	2
noop_detection_social_network	1
noop_detection_astronomy_shop	1
container_kill	2
Test-only fault types subtotal	*48*
+ Training fault types (Sonnet failed)	*15*
Total held-out	**63**

C.1 Diminishing Returns in Sampling

Figure 7 shows success rate improvement across multiple sampling rounds.

Practical deployment guidance. The steep improvement from best@1 (31.4%) to best@2 (51.2%) suggests that even a single retry captures most of the "easy wins" from stochastic variation. Beyond k=3, gains become marginal (best@3=58.1% → best@5=66.3%). For cost-sensitive deployments, we recommend k=2 as the optimal trade-off; for high-stakes scenarios where maximizing success rate justifies additional compute, k=3 captures 88% of the maximum achievable improvement.

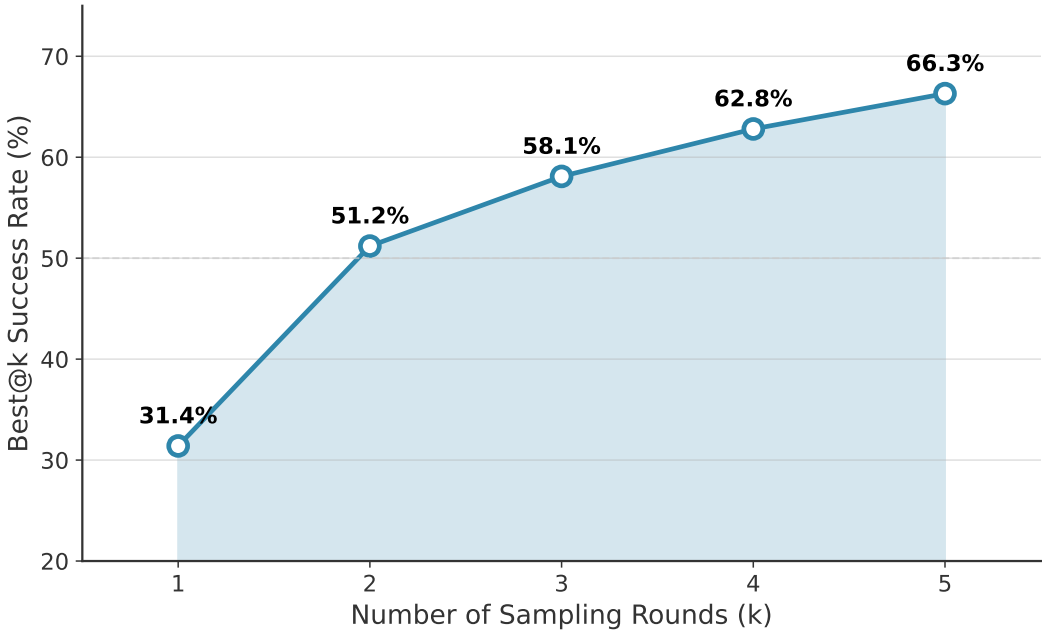

Figure 7. best@k success rate vs number of runs. The curve shows clear diminishing returns: the first retry (k=1→2) provides +19.8pp improvement, while subsequent retries (k=2→5) each provide only +2-3pp. *Interpretation:* The rapid initial improvement suggests many tasks have "near-miss" failure modes that succeed on retry, while the flattening curve indicates some tasks are fundamentally beyond current capability. The inflection point at k=2-3 provides clear guidance for resource-constrained deployments.

C.2 Per-Round Success Rates

Table 8 shows success rates for each of the 5 independent runs, broken down by task type.

Table 8. Per-round success rates (%) by task type.

Round	Det.	Loc.	RCA	Mit.	Overall
R1	53.1	21.4	7.7	23.1	31.4
R2	75.0	32.1	0.0	23.1	41.9
R3	59.4	32.1	15.4	23.1	38.4
R4	75.0	21.4	15.4	23.1	40.7
R5	71.9	32.1	0.0	23.1	40.7
Avg	66.9	27.9	7.7	23.1	38.6

Observations: Detection shows high variance (53–75%), indicating significant benefit from retries. Mitigation is perfectly stable (23.1% every round), suggesting that failures are capability-limited rather than stochastic. RCA is highly unstable (0–15.4%) due to small sample size (13 tasks) and complex multi-step reasoning requirements.

Figure 8 visualizes this variance.

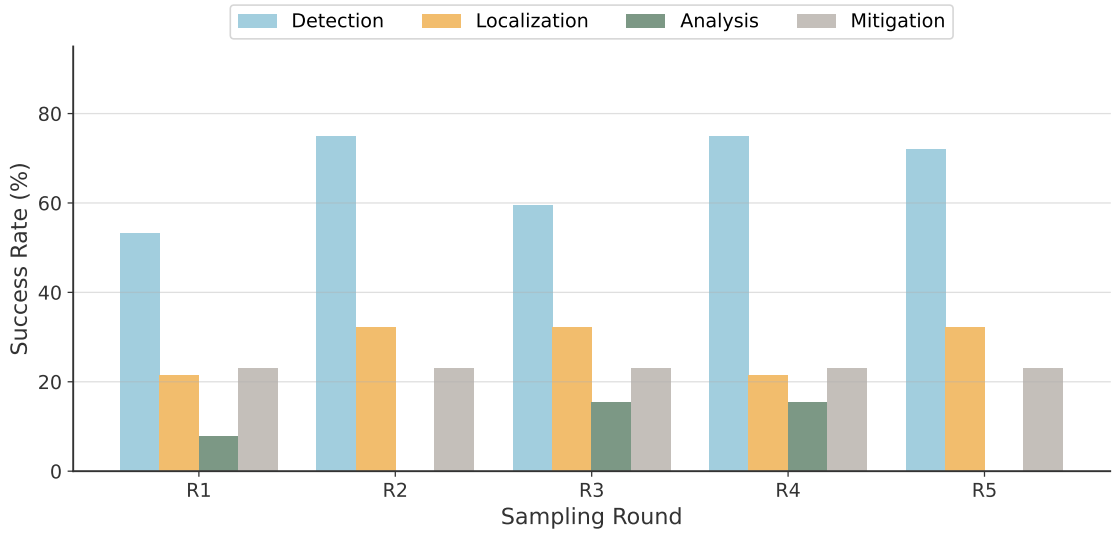

Figure 8. Per-round success rate variance by task type. Detection exhibits the widest range (22pp), indicating high stochastic sensitivity—tasks that fail in one run often succeed in another due to different exploration paths. Mitigation's flat line confirms that current failures are deterministic capability gaps rather than sampling artifacts. *Actionable insight:* Detection tasks benefit most from multi-round sampling, while Mitigation requires architectural improvements rather than more attempts.

C.3 Task Stability Distribution

We categorize tasks by their 5-round success count to understand task difficulty distribution (Table 9).

Table 9. Task stability distribution (86 tasks total).

Success	Category	Count	%
5/5	Consistently solved	14	16.3
3–4/5	Mostly solved	16	18.6
1–2/5	Occasionally solved	27	31.4
0/5	Never solved	29	33.7

Key insight: 50% of tasks (1–4/5) exhibit stochastic outcomes, strongly validating multi-round sampling. The 29 never-solved tasks represent hard capability boundaries requiring architectural improvements rather than more sampling. The 14 always-solved tasks demonstrate the system's reliable core competencies.

Figure 9 visualizes this distribution.

C.4 Never-Solved Tasks (0/5)

The 29 consistently failed tasks cluster by type:

- **Localization (13):** Primarily astronomy_shop microservice dependencies requiring deep trace analysis beyond current model capabilities
- **RCA (9):** Causal reasoning tasks requiring understanding of temporal fault propagation
- **Mitigation (7):** Domain-specific remediation commands (e.g., MongoDB auth recovery, Helm chart upgrades) not covered in training data

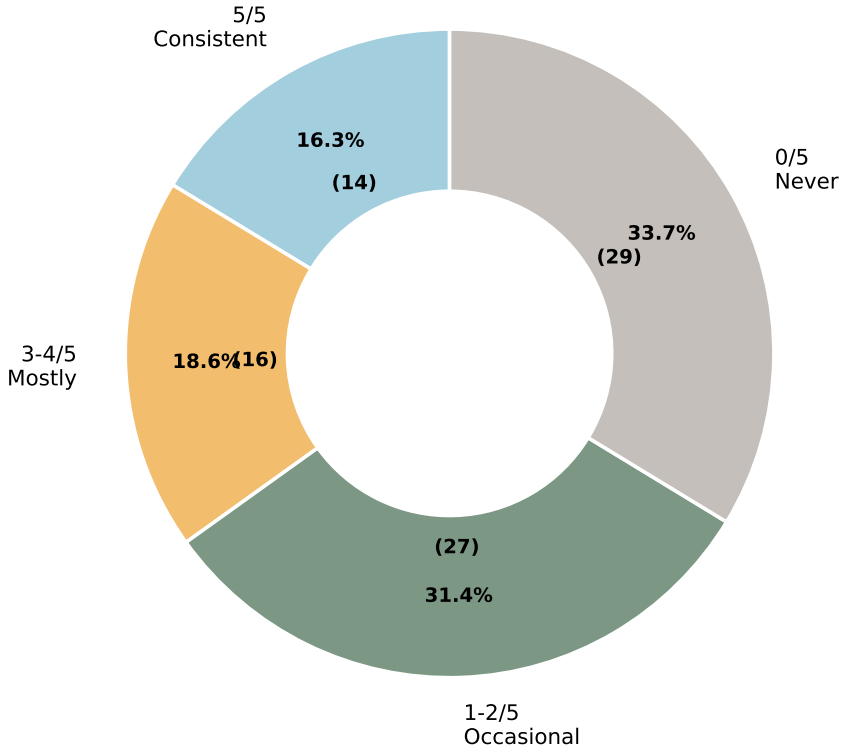

Figure 9. Task stability distribution across 86 benchmark tasks. The large middle segment (50% of tasks with 1-4/5 success) validates our multi-round sampling strategy—these tasks are neither trivially easy nor impossibly hard, but rather exhibit outcome variance across runs. *Design implication:* Future work should focus on converting the 0/5 category (capability gaps) into the 1-2/5 category (stochastic successes), which can then be reliably solved via multi-round sampling.

D Observer GRPO Analysis

Organization note: This section shifts from characterizing system behavior (Section C) to analyzing the *effect of GRPO training*. We group all GRPO-related analysis together (Sections D and E) to provide a cohesive view of how training improves (and sometimes degrades) performance across different task types.

D.1 Overall Training Effect

Figure 10 shows the aggregate impact of Observer GRPO training across task types.

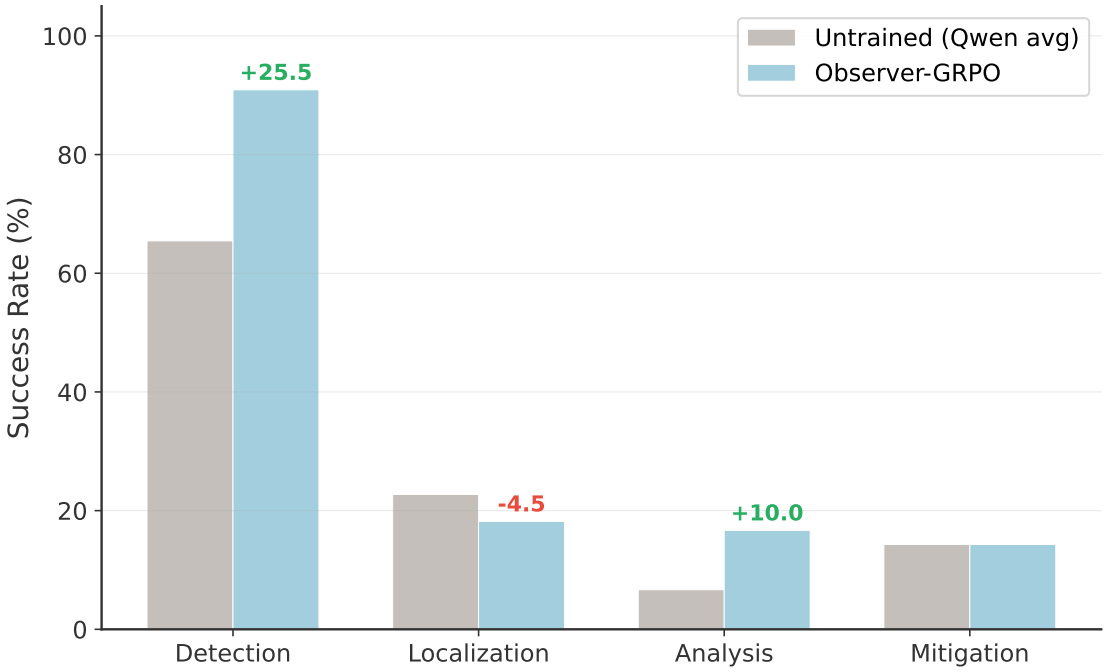

Figure 10. Observer GRPO training effect by task type (best@5). Detection improves dramatically (+25.5pp) as the Observer learns systematic anomaly-detection patterns. Localization *decreases* (−4.5pp) due to over-exploration causing multi-anomaly confusion (detailed in Section E.4). RCA improves (+8.4pp) from better evidence-gathering strategies. *Critical insight:* GRPO training is not uniformly beneficial—it helps exploration-heavy tasks but can harm precision-critical tasks where thoroughness conflicts with efficiency.

D.2 Task-Level Changes

Figure 11 shows how individual tasks changed after GRPO training.

D.3 Improved Tasks (11)

Tasks that changed from mostly-failing (Qwen <50%) to successful after GRPO:

- **Detection (7):** payment_service_unreachable, ad_service_failure, wrong_bin_usage, kafka_queue_problems, redeploy_without_PV, cart_service_failure, network_delay_hotel_res

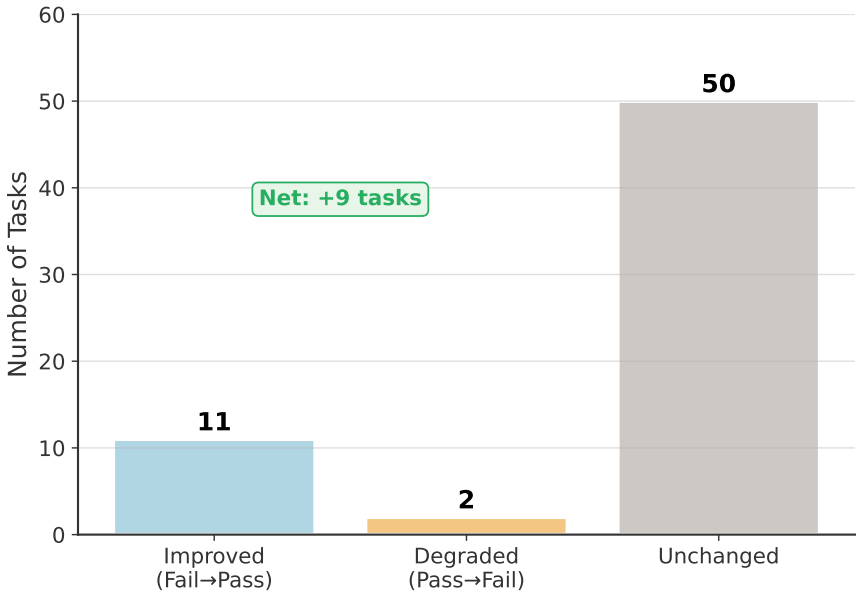

Figure 11. Task-level changes after GRPO training. Net +9 tasks improved (11 gains, 2 losses). Both degraded tasks are Localization, confirming that GRPO's deep-exploration strategy harms precision-critical fault localization. *Interpretation:* The asymmetric gains (strong Detection improvements, Localization degradations) suggest task-type-aware training as future work.

- **Localization (2):** container_kill, network_delay_hotel_res
- **RCA (2):** k8s_target_port-misconfig-analysis-2, revoke_auth_mongodb-analysis-2

D.4 Degraded Tasks (2)

Both degraded tasks are Localization:

- pod_failure_hotel_res-localization-1 (4/5 → 0/5)
- product_catalog_service_failure-localization-1 (4/5 → 0/5)

Root cause analysis. Both degraded tasks require identifying a specific faulty component among multiple candidates with similar symptoms. Before GRPO, the base model would conservatively check only the most obvious candidate; after GRPO, the Observer learns to explore extensively—a strategy that succeeds for Detection (where finding *any* issue suffices) but fails for Localization (where the *exact* root cause must be identified). This reveals a fundamental tension: GRPO's reward signal optimizes for task completion, which can conflict with the thoroughness required for precise component identification.

E Observer-GRPO vs Base: Comprehensive Task-Type Analysis

This section provides detailed analysis of how GRPO training affects performance across different task types, revealing a critical trade-off between exploration depth and task requirements.

E.1 Training Set Distribution

The Observer GRPO training uses 23 successful trajectories with the following task-type distribution:

Table 10. Training set composition by task type. The heavy Detection bias (43.5%) explains why Detection benefits most from GRPO.

Task Type	Training Tasks	Test Tasks
Detection	10 (43.5%)	22
Localization	6 (26.1%)	22
Analysis (RCA)	1 (4.3%)	12
Mitigation	6 (26.1%)	7
Total	**23**	**63**

Critical observation: Analysis tasks are severely underrepresented in training (only 1 task, 4.3%), yet Observer-GRPO shows large improvement on Analysis (+16.7pp). This suggests the learned exploration patterns transfer well despite limited direct supervision.

E.2 Performance Comparison Overview

Figure 12 visualizes the performance gap between Observer-GRPO and Base models.

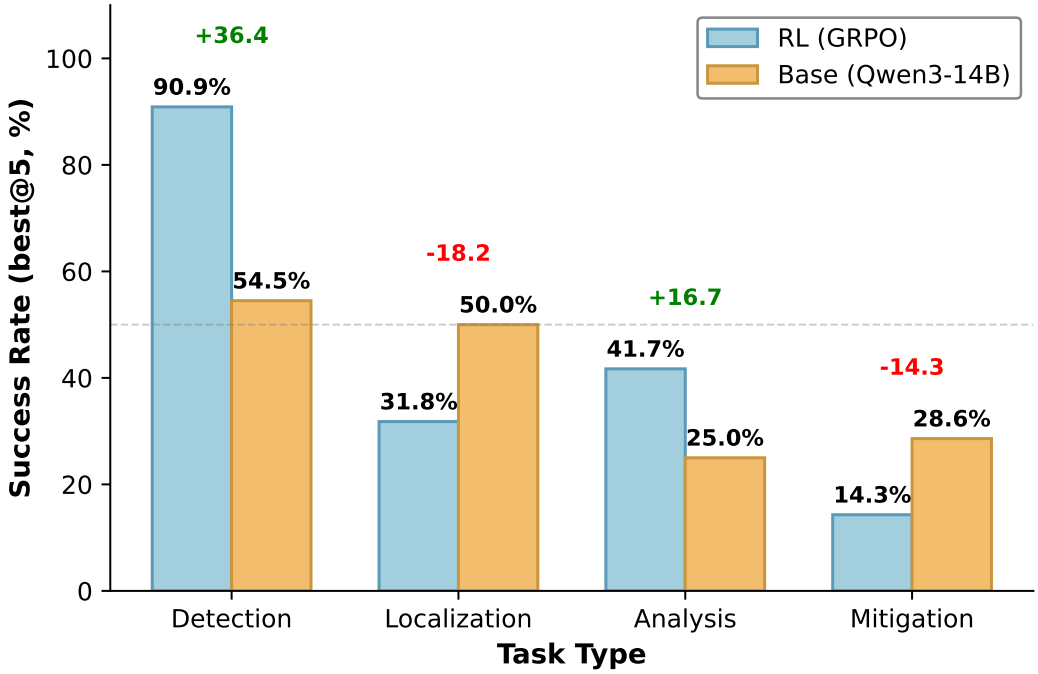

Figure 12. Observer-GRPO vs Base performance by task type (best@5). Detection improves by +36.4pp (from 59.1% to 95.5%), while Localization *degrades* by −18.2pp (50.0% to 31.8%). *Key takeaway:* GRPO benefits evidence-gathering tasks (Detection, Analysis) but harms precision tasks (Localization) where over-exploration introduces confusion. This heterogeneity motivates task-type-aware exploration policies.

E.3 Exploration Depth Analysis

Table 11 compares the average number of exploration steps between Observer-GRPO and Base models.

Table 11. Average exploration steps by task type (lower = more efficient).

Task Type	Obs-GRPO	Base	Diff.
Detection	11.0	1.9	+9.1
Localization	11.2	1.8	+9.4
Analysis	9.6	1.1	+8.5
Mitigation	12.0	2.8	+9.2
Average	**10.9**	**1.9**	**+9.0**

Observer-GRPO consistently uses ∼9 more steps than Base across all task types. This "deep exploration" strategy has task-type-dependent effects:

- **Detection**: *Beneficial*—deeper search finds anomalies Base misses
- **Localization**: *Harmful*—over-exploration discovers multiple anomalies, causing incorrect root cause selection
- **Analysis**: *Beneficial*—deeper exploration gathers sufficient evidence for correct fault classification
- **Mitigation**: Neutral effect—both struggle with execution-heavy tasks

Figure 13 visualizes the relationship between exploration depth and success rate.

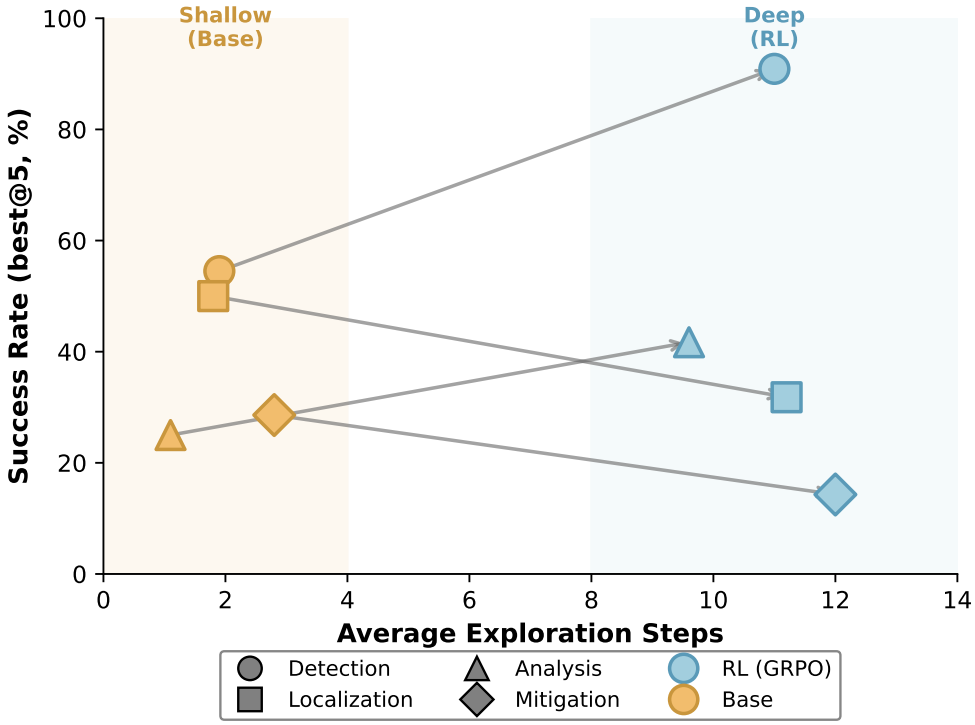

Figure 13. Exploration depth vs success rate trade-off. Each point represents a task type; arrows show the transition from Base to Observer-GRPO. GRPO uniformly increases exploration depth (arrows point right), but success direction varies: upward for Detection/Analysis (more exploration helps), downward for Localization (more exploration harms). *Critical implication:* A single exploration strategy cannot optimize all task types simultaneously—future work should adapt exploration depth based on detected task category.

E.4 Localization: Why Observer-GRPO Underperforms

Table 12 shows the detailed comparison for Localization tasks.

Table 12. Localization task outcomes (22 tasks total).

Outcome	Count	%
Obs-GRPO wins (Base fails)	2	9.1%
Base wins (Obs-GRPO fails)	6	27.3%
Both succeed	5	22.7%
Both fail	9	40.9%
best@5	Obs-GRPO: 31.8% vs Base: 50.0%	

Obs-GRPO wins: container_kill, network_loss_hotel_res.
Base wins: ad_service_failure, ad_service_high_cpu, payment_service_failure, k8s_misconfig-loc-2, network_delay, user_unregistered-loc-2.

Root cause of Observer-GRPO degradation: In the 6 tasks where Base wins:

1. Observer-GRPO explores extensively (∼10 steps), discovering multiple components with anomalies
2. When multiple anomalies exist, Observer-GRPO often submits the *symptom* rather than the *root cause*
3. Base's shallow exploration (1–2 steps) paradoxically helps by limiting the search to the most obvious fault

Case study: astronomy_shop_ad_service_failure-localization-1

- **Ground truth**: The `ad` service is the root cause
- **Base** (2 steps): Quickly identifies `ad` service errors → submits `["ad"]` → **Correct**
- **Obs-GRPO** (10 steps): Finds `ad` errors, explores dependencies, finds `product-catalog` affected → submits `["product-catalog"]` → **Incorrect** (downstream symptom, not root cause)

E.5 Analysis: Why Observer-GRPO Excels

Table 13 shows the detailed comparison for Analysis (RCA) tasks.

Table 13. Analysis task outcomes (12 tasks total).

Outcome	Count	%
Obs-GRPO wins (Base fails)	2	16.7%
Base wins (Obs-GRPO fails)	0	0.0%
Both succeed	3	25.0%
Both fail	7	58.3%
best@5	Obs-GRPO: 41.7% vs Base: 25.0%	

Obs-GRPO wins: k8s_misconfig-analysis-1, revoke_auth-analysis-1.
Both succeed: k8s_misconfig-analysis-2, misconfig_app, revoke_auth-analysis-2.

Why deeper exploration helps Analysis: Analysis tasks require determining both `system_level` (e.g., Application, Virtualization, Network) and `fault_type` (e.g., Misconfiguration, Resource Exhaustion). This classification requires:

1. Understanding which layer the fault originates from
2. Gathering evidence about the nature of the fault
3. Eliminating alternative hypotheses

Base's shallow exploration (avg 1.1 steps) provides insufficient evidence for reliable classification. Observer-GRPO's deeper exploration (avg 9.6 steps) systematically gathers the necessary information.

Case study: k8s_target_port-misconfig-analysis-1

- **Ground truth**: `system_level: Virtualization, fault_type: Misconfiguration`
- **Base** (1.6 steps avg): Insufficient exploration → 0/5 rounds correct
- **Obs-GRPO** (10 steps): Examines service definitions, finds `targetPort` mismatch → identifies Virtualization Misconfiguration → 4/5 correct

E.6 Task Outcome Distribution

Figure 14 compares task-level outcomes between Observer-GRPO and Base for Localization and Analysis.

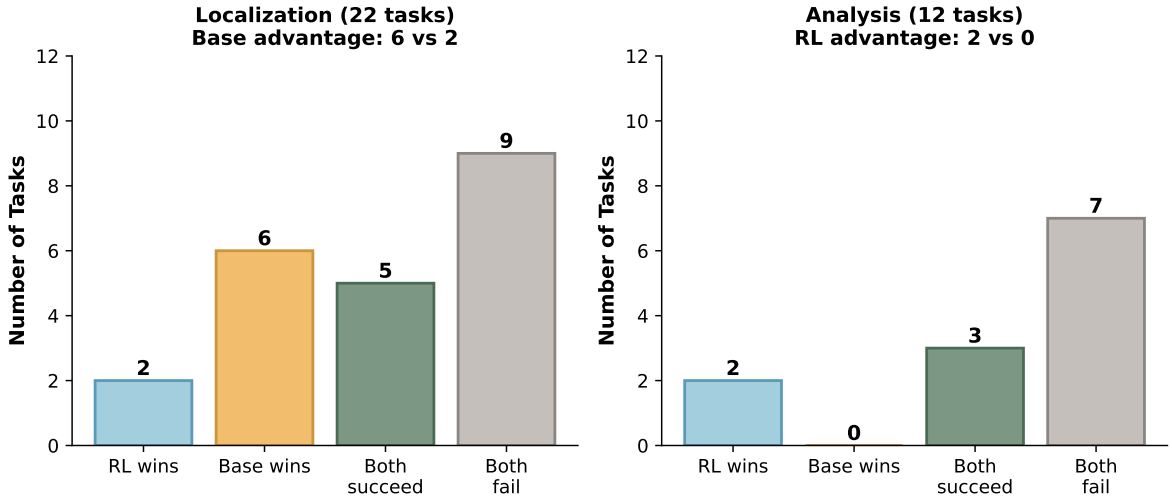

Figure 14. Task outcome distribution for Localization vs Analysis. Localization shows Base advantage (6 exclusive wins vs 2), while Analysis shows Obs-GRPO advantage (2 vs 0). *Interpretation:* Confirms that deep exploration helps Analysis (which benefits from comprehensive evidence) but harms Localization (where first instinct is often correct). The asymmetry validates task-type-specific exploration strategies.

E.7 The Exploration-Precision Trade-off

Our analysis reveals a fundamental trade-off in diagnostic agent design:

Table 14. Exploration depth trade-off by task type.

Task Type	Optimal Strategy	Why
Detection	Shallow OK	Any anomaly suffices
Localization	Shallow better	Avoid multi-anomaly confusion
Analysis	Deep better	Need comprehensive evidence
Mitigation	Deep required	Must understand before fixing

Implications for future work:

1. **Task-aware exploration**: Dynamically adjust exploration depth based on detected task type
2. **Confidence-based early stopping**: For Localization, stop when a high-confidence root cause is found

3. **Multi-stage training**: Train separate policies for different task types, or use task-type-conditioned rewards

Figure 15 provides a visual summary.

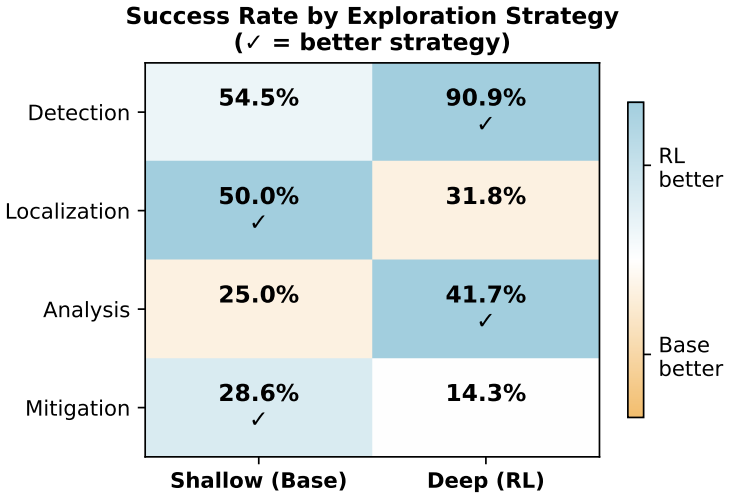

Figure 15. Exploration strategy effectiveness heatmap. Green = strategy helps; Red = strategy harms. Localization uniquely benefits from shallow exploration, while Analysis requires deep exploration. This heterogeneity strongly motivates task-type-aware policies as future work.

F Case Studies: Detailed Trajectory Examples

Organization note: After presenting quantitative results (Sections C-E), this section provides *qualitative illustration* through detailed execution traces. We include multiple case studies covering different task types and failure modes to demonstrate how the multi-agent architecture operates in practice. Each case study highlights specific design decisions (read-write separation, dual-timescale memory, Evolver guidance) in action.

F.1 Case Study: Mitigation with Storage Class Failure

Why this case: This example illustrates the complete AOI pipeline—Observer reasoning, Probe exploration, Executor remediation, and verification. It demonstrates: (1) how the Observer identifies root causes through systematic exploration; (2) how read-write separation prevents premature state mutations; (3) how the Executor applies fixes only after sufficient evidence accumulates.

Scenario: `redeploy_without_PV-mitigation-1`. The StorageClass for persistent volumes was removed during redeployment, causing cascading PVC binding failures and pod crashes.

Phase: Initialization Run 1

```
[04:00:28] Fault injected:  StorageClass 'local-storage' deleted
[04:00:29] Namespace 'test-hotel-reservation' redeployed without PV
[04:00:35] Multiple pods entering CrashLoopBackOff state
Task Started
```

Thought & Action — Observer Agent

The task requires mitigating anomalies in test-hotel-reservation. I should first understand the current state of the system by listing all resources in the namespace.

Action: Probe

Subtask: "List all pods/services in namespace to identify failing components"

Tool Usage: kubectl — Probe Agent

```
$ kubectl get pods -n test-hotel-reservation
NAME                              READY     STATUS              RESTARTS
consul-b84877fb4-v9sgv       1/1      Running           0
geo-99d8c58c-5wrbn           0/1      CrashLoopBackOff   3
mongodb-geo-5ff578bcb8       0/1      Pending            0
mongodb-profile-758cb        0/1      Pending            0
reservation-78c9f79          0/1      CrashLoopBackOff   5
...  (8 more pods in error state)
```

Thought & Action — Observer Agent

Multiple pods are failing. The mongodb pods are in Pending state while service pods are in CrashLoopBackOff. This suggests a volume mounting issue. I need to check the PVC status.

Action: Probe

Subtask: "Check PVC status and describe pending pods"

Tool Usage: kubectl — Probe Agent

```
$ kubectl get pvc -n test-hotel-reservation
NAME                  STATUS      STORAGECLASS
mongodb-geo-pvc      Pending     local-storage
mongodb-profile-pvc   Pending     local-storage
mongodb-rate-pvc      Pending     local-storage

$ kubectl describe pvc mongodb-geo-pvc -n test-hotel-reservation
Events:
Warning    ProvisioningFailed    storageclass.storage.k8s.io
'local-storage' not found
```

Thought & Action — Observer Agent

Root cause identified: The StorageClass "local-storage" is missing, preventing PVCs from binding. The mongodb pods cannot start without their persistent volumes, and dependent services fail. **Solution:** Recreate the StorageClass, then trigger PVC rebinding.

Action: Execute

Instruction: "Create StorageClass 'local-storage' with provisioner 'kubernetes.io/no-provisioner' and volumeBindingMode 'WaitForFirstConsumer"'

Tool Usage: kubectl apply — Executor Agent

```
$ kubectl apply -f storageclass.yaml

apiVersion:  storage.k8s.io/v1
kind:  StorageClass
metadata:  name:  local-storage
provisioner:  kubernetes.io/no-provisioner

storageclass.storage.k8s.io/local-storage created
```

Thought & Action — Observer Agent

StorageClass recreated. Now I need to delete the stuck PVCs and let them rebind, then restart the deployments to pick up the changes.
Action: Execute
Instruction: "Delete pending PVCs and restart all deployments"

Tool Usage: kubectl — Executor Agent

```
$ kubectl delete pvc -all -n test-hotel-reservation
persistentvolumeclaim 'mongodb-geo-pvc' deleted
persistentvolumeclaim 'mongodb-profile-pvc' deleted
persistentvolumeclaim 'mongodb-rate-pvc' deleted

$ kubectl rollout restart deployment -all -n test-hotel-reservation
deployment.apps/consul restarted
deployment.apps/geo restarted
deployment.apps/mongodb-geo restarted
...  (9 more deployments restarted)
```

Thought & Action — Observer Agent

Mitigation commands executed. Let me verify that all pods are now running correctly before submitting.
Action: Probe
Subtask: "Verify all pods are Running"

Tool Usage: kubectl — Probe Agent

```
$ kubectl get pods -n test-hotel-reservation
NAME                              READY     STATUS      RESTARTS
consul-b84877fb4-x2kpl       1/1      Running     0
geo-99d8c58c-7mnqr           1/1      Running     0
mongodb-geo-5ff578b-new     1/1      Running     0
mongodb-profile-758cb-new   1/1      Running     0
reservation-78c9f79-new      1/1      Running     0
...  (all 12 pods Running)
```

Task Finished — Observer Agent

Diagnosis: StorageClass "local-storage" was missing after redeployment, causing PVC binding failures and cascading pod failures. **Mitigation:** (1) Recreated StorageClass with correct provisioner; (2) Deleted stuck PVCs to trigger rebinding; (3) Restarted all deployments to pick up new PVCs. **Result:** All 12 pods now Running. Services operational.

Phase: Validation — Run 1

```
[04:19:08] Validation started...
[04:19:10] Pod check:  12/12 Running PASS
[04:19:12] Service check:  All endpoints healthy PASS
[04:19:15] Task completed successfully
Total time:  1139 seconds, 13 iterations
```

Key takeaways from this case:

- **Systematic exploration:** Observer first gathers evidence (pod status, PVC status) before taking action
- **Read-write separation:** All diagnostic commands execute through Probe; only after root cause is confirmed does Executor engage
- **Verification loop:** After applying fixes, Observer verifies success before submitting
- **Multi-step reasoning:** Recognizes that fixing StorageClass alone isn't enough—must also trigger PVC rebinding and deployment restart

G Detailed Per-Task Results

Organization note: This final section provides the complete empirical record of all 86 tasks across 5 runs. We place this at the end because it serves as a reference appendix rather than primary analysis. Researchers can use these tables to identify specific tasks for further investigation or to verify aggregate statistics reported in the main paper.

Tables 15–18 present the complete per-task results across 5 independent runs. **T** indicates success, **F** indicates failure.

Table 15. Detailed results: **Detection** tasks (32 total). High variance across runs indicates stochastic sensitivity to exploration paths.

Problem ID	R1	R2	R3	R4	R5	Best
assign_to_non_existent_node-det-1	T	T	T	T	T	5/5
astronomy_shop_ad_service_failure-det-1	F	T	F	T	F	2/5
astronomy_shop_ad_service_high_cpu-det-1	T	T	F	T	T	4/5
astronomy_shop_ad_service_manual_gc-det-1	F	T	T	T	F	3/5
astronomy_shop_cart_service_failure-det-1	F	T	F	T	F	2/5
astronomy_shop_image_slow_load-det-1	F	T	T	T	T	4/5
astronomy_shop_kafka_queue_problems-det-1	F	T	T	F	F	2/5
astronomy_shop_loadgen_flood-det-1	T	T	T	T	T	5/5
astronomy_shop_payment_failure-det-1	F	T	T	T	F	3/5
astronomy_shop_payment_unreachable-det-1	F	T	F	T	F	2/5
astronomy_shop_product_catalog-det-1	T	T	T	T	T	5/5
astronomy_shop_recommend_cache-det-1	T	T	F	F	T	3/5
auth_miss_mongodb-det-1	T	T	T	T	T	5/5
container_kill-det	T	F	F	T	T	3/5
k8s_target_port-misconfig-det-1	T	F	F	F	F	1/5
k8s_target_port-misconfig-det-2	F	F	F	T	T	2/5
k8s_target_port-misconfig-det-3	F	T	T	T	T	4/5
misconfig_app_hotel_res-det-1	T	T	T	T	T	5/5
network_delay_hotel_res-det-1	F	F	T	F	T	2/5
network_loss_hotel_res-det-1	F	F	F	F	T	1/5
noop_detection_astronomy_shop-1	T	F	F	F	T	2/5
noop_detection_hotel_reservation-1	T	T	T	F	T	4/5
noop_detection_social_network-1	F	T	T	T	T	4/5
pod_failure_hotel_res-det-1	T	T	T	T	T	5/5
pod_kill_hotel_res-det-1	F	F	F	F	T	1/5
redeploy_without_PV-det-1	F	F	F	T	F	1/5
revoke_auth_mongodb-det-1	T	T	T	T	T	5/5
revoke_auth_mongodb-det-2	T	T	T	T	T	5/5
scale_pod_zero_social_net-det-1	T	T	T	T	T	5/5
user_unregistered_mongodb-det-1	T	T	T	T	T	5/5
user_unregistered_mongodb-det-2	T	T	T	T	T	5/5
wrong_bin_usage-det-1	F	T	F	T	F	2/5
Total	17/32	24/32	19/32	24/32	23/32	–

Table 16. Detailed results: **Localization** tasks (28 total).

Problem ID	R1	R2	R3	R4	R5	Best
assign_to_non_existent_node-loc-1	T	T	T	T	T	5/5
astronomy_shop_ad_service_failure-loc-1	F	T	F	F	F	1/5
astronomy_shop_ad_service_high_cpu-loc-1	F	T	F	F	T	2/5
astronomy_shop_ad_service_manual_gc-loc-1	F	F	F	F	F	0/5
astronomy_shop_cart_service_failure-loc-1	F	F	F	F	F	0/5
astronomy_shop_image_slow_load-loc-1	F	F	F	F	F	0/5
astronomy_shop_kafka_queue_problems-loc-1	F	F	F	F	F	0/5
astronomy_shop_loadgen_flood-loc-1	F	F	F	F	F	0/5
astronomy_shop_payment_failure-loc-1	F	F	F	F	T	1/5
astronomy_shop_payment_unreachable-loc-1	F	F	F	F	F	0/5
astronomy_shop_product_catalog-loc-1	T	T	T	F	T	4/5
astronomy_shop_recommend_cache-loc-1	F	F	F	F	F	0/5
auth_miss_mongodb-loc-1	F	F	T	T	T	3/5
container_kill-loc	F	F	F	F	F	0/5
k8s_target_port-misconfig-loc-1	T	F	T	T	F	3/5
k8s_target_port-misconfig-loc-2	F	T	F	F	T	2/5
k8s_target_port-misconfig-loc-3	T	T	T	F	F	3/5
misconfig_app_hotel_res-loc-1	T	F	T	T	T	4/5
network_delay_hotel_res-loc-1	F	T	F	F	F	1/5
network_loss_hotel_res-loc-1	F	F	F	F	F	0/5
pod_failure_hotel_res-loc-1	F	T	T	T	T	4/5
pod_kill_hotel_res-loc-1	F	F	F	F	F	0/5
revoke_auth_mongodb-loc-1	T	F	F	F	F	1/5
revoke_auth_mongodb-loc-2	F	F	F	F	F	0/5
scale_pod_zero_social_net-loc-1	F	T	T	T	T	4/5
user_unregistered_mongodb-loc-1	F	F	F	F	F	0/5
user_unregistered_mongodb-loc-2	F	F	T	F	F	1/5
wrong_bin_usage-loc-1	F	F	F	F	F	0/5
Total	6/28	9/28	9/28	6/28	9/28	–

Table 17. Detailed results: **RCA** tasks (13 total).

Problem ID	R1	R2	R3	R4	R5	Best
assign_to_non_existent_node-rca-1	F	F	F	F	F	0/5
auth_miss_mongodb-rca-1	F	F	F	F	F	0/5
k8s_target_port-misconfig-rca-1	F	F	F	F	F	0/5
k8s_target_port-misconfig-rca-2	T	F	F	F	F	1/5
k8s_target_port-misconfig-rca-3	F	F	T	F	F	1/5
misconfig_app_hotel_res-rca-1	F	F	F	T	F	1/5
redeploy_without_PV-rca-1	F	F	F	F	F	0/5
revoke_auth_mongodb-rca-1	F	F	F	F	F	0/5
revoke_auth_mongodb-rca-2	F	F	T	T	F	2/5
scale_pod_zero_social_net-rca-1	F	F	F	F	F	0/5
user_unregistered_mongodb-rca-1	F	F	F	F	F	0/5
user_unregistered_mongodb-rca-2	F	F	F	F	F	0/5
wrong_bin_usage-rca-1	F	F	F	F	F	0/5
Total	1/13	0/13	2/13	2/13	0/13	–

Table 18. Detailed results: **Mitigation** tasks (13 total).

Problem ID	R1	R2	R3	R4	R5	Best
assign_to_non_existent_node-mit-1	T	T	T	T	T	5/5
auth_miss_mongodb-mit-1	F	F	F	F	F	0/5
k8s_target_port-misconfig-mit-1	T	F	F	F	F	1/5
k8s_target_port-misconfig-mit-2	F	F	F	T	F	1/5
k8s_target_port-misconfig-mit-3	F	F	T	F	F	1/5
misconfig_app_hotel_res-mit-1	F	F	F	F	F	0/5
redeploy_without_PV-mit-1	T	T	T	T	T	5/5
revoke_auth_mongodb-mit-1	F	F	F	F	F	0/5
revoke_auth_mongodb-mit-2	F	F	F	F	F	0/5
scale_pod_zero_social_net-mit-1	F	T	F	F	T	2/5
user_unregistered_mongodb-mit-1	F	F	F	F	F	0/5
user_unregistered_mongodb-mit-2	F	F	F	F	F	0/5
wrong_bin_usage-mit-1	F	F	F	F	F	0/5
Total	3/13	3/13	3/13	3/13	3/13	–